ENASTIC
Ecole Nationale Supérieure des TIC

Projet de Fin d'Études

En vue de l'obtention du **Diplôme de Licence en Informatique**

THÈME

CyberSentinel-LLM :

Plateforme pour la Détection Contextuelle et la Neutralisation Adaptive des Menaces Réseau

Présenté par :

M. YAKHOUB MAHAMAT HAMDO

Encadrant :

Dr. MOUAZ MICKAIL

Enseignant chercheur en IA

Jury :

Président : M. _____

Rapporteur : M. _____

Année Universitaire : 2025–2026

AVANT-PROPOS

Fondée par l'Ordonnance **N°005/PR/2015** du 02 mars 2015, l'École Nationale Supérieure des Technologies de l'Information et de la Communication (ENASTIC) est une institution publique dotée de la personnalité juridique et de l'autonomie financière, dont la vocation centrale est de former des cadres de haut niveau dans le secteur numérique.

Opérant sous la double tutelle du Ministère des Postes et des Nouvelles Technologies de l'Information et de la Communication d'une part, et du Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation d'autre part, l'ENASTIC poursuit trois grandes missions :

- ☞ Dispenser une formation de qualité, initiale comme continue, destinée aux futurs cadres du domaine des TIC ;
- ☞ Conduire des travaux de recherche appliquée à vocation nationale, contribuant au développement socio-économique du Tchad ;
- ☞ Cultiver chez les étudiants un esprit scientifique et entrepreneurial, les rendant capables de concevoir et de porter des projets innovants à fort impact numérique.

La réalisation d'un Projet de Fin d'Études (PFE), soutenu devant un jury académique, constitue l'étape finale et obligatoire du cycle de Licence. C'est dans ce cadre que s'inscrit le présent mémoire, rédigé au titre de l'année universitaire 2025–2026, sous le thème : **« CYBERSENTINEL-LLM : PLATEFORME POUR LA DÉTECTION CONTEXTUELLE ET LA NEUTRALISATION ADAPTIVE DES MENACES RÉSEAU »**.

Ce document décrit la conception et le développement d'un prototype fonctionnel croisant trois axes techniques complémentaires : la détection d'anomalies réseau par des algorithmes de Machine Learning non supervisés, le raisonnement contextuel d'un grand modèle de langage local enrichi par la base MITRE ATT&CK via une approche RAG, et la neutralisation des menaces par blocage applicatif conditionné à une approbation humaine explicite. Le tout vise à outiller concrètement les analystes SOC dans leur tâche quotidienne d'interprétation, de priorisation et de traitement des incidents de sécurité, dans un contexte de ressources locales limitées comme celui du Tchad.

DÉDICACE

Je dédie ce modeste travail à mes parents.

À mon défunt père, qui a su me soutenir jusqu'à ses derniers instants
et aussi à ma mère qui est notre pilier et qui demeure mon modèle.

tout ce que j'entreprends est pour eux.

REMERCIEMENTS

Ce mémoire n'aurait pas vu le jour sans le concours et l'appui de nombreuses personnes que je souhaite remercier ici de tout cœur.

Ma reconnaissance va tout particulièrement à :

- ☞ Ma famille et mes proches, pour leur soutien moral et leur encouragement constants tout au long de ce parcours ;
- ☞ L'équipe pédagogique de l'École Nationale Supérieure des Technologies de l'Information et de la Communication (ENASTIC), pour la qualité de la formation dispensée ;
- ☞ Mon Directeur de mémoire, pour la qualité de son encadrement, sa disponibilité et ses précieux conseils tout au long de la réalisation de ce projet.

CONTENTS

Avant-propos I

Dédicace II

Remerciements III

Liste des figures V

Liste des tableaux VI

Liste des Abréviations VII

Introduction Générale 1

1 Généralités 4
Introduction . . . 4
1.1 Présentation des IDS et IPS . . . 4
1.2 Limites des approches classiques . . . 5
1.3 Apport du Machine Learning en détection d'anomalies . . . 5
1.4 Apport des LLM et principe Human-in-the-Loop . . . 6
1.5 RAG pour la cybersécurité . . . 7
Conclusion . . . 7

2 Analyse des Besoins et Spécifications 8
Introduction . . . 8
2.1 Cas d'usage principaux . . . 8
2.2 Exigences fonctionnelles . . . 9
2.3 Exigences non fonctionnelles . . . 10
Conclusion . . . 11

3 Conception Logicielle et Architecture 12
Introduction . . . 12
3.1 Architecture globale . . . 12
3.2 Arborescence du projet . . . 13
3.3 Description des composants backend . . . 14
3.4 Description du frontend . . . 15
3.5 Refactoring frontend — Architecture modulaire . . . 16

3.6 Diagrammes UML . . . 17
3.6.1 Diagramme de classes . . . 17
3.6.2 Diagramme de séquence . . . 18
3.6.3 Diagramme d'activité . . . 19
Conclusion . . . 20

4 Implémentation **21**
Introduction . . . 21
4.1 Pipeline Machine Learning . . . 21
4.2 Module RAG — MITRE ATT&CK . . . 22
4.3 Module LLM et Score de Confiance . . . 24
4.4 Système de Neutralisation Adaptive . . . 25
4.5 Endpoints principaux de l'API . . . 27
4.6 SSE et communication temps réel . . . 27
4.7 Architecture frontend temps réel . . . 28
4.8 Interfaces utilisateur . . . 29
Conclusion . . . 33

5 Tests, Validation et Résultats **34**
Introduction . . . 34
5.1 Scénario de démonstration complet . . . 34
5.2 Mécanismes d'évaluation . . . 36
5.3 Résultats observés . . . 36
5.4 Tests automatisés . . . 37
Conclusion . . . 39

Conclusion Générale et Perspectives **40**

Bibliographie **42**

Annexes **43**
Annexe A : Architecture globale . . . 43
Annexe B : Exemples de payloads API . . . 44
Annexe C : Guide d'installation . . . 45

LIST OF FIGURES

2.1 Diagramme de cas d'utilisation de CyberSentinel-LLM 9

3.1 Architecture globale de CyberSentinel-LLM 13
3.2 Diagramme de classes de CyberSentinel-LLM 18
3.3 Diagramme de séquence — flux d'une alerte jusqu'à la neutralisation 19
3.4 Diagramme d'activité — traitement d'un log entrant 20

4.1 Tableau de bord SOC — KPIs temps réel, timeline 24h et carte GeoIP 29
4.2 Page Alertes — liste complète avec labellisation et gestion des blocages 29
4.3 Page Incidents — détail d'un incident avec explication LLM structurée 30
4.4 Page Chat SOC — console conversationnelle streaming avec historique 30
4.5 Page Paramètres — état des services, 2FA TOTP, gestion des utilisateurs et IPs bloquées 31
4.6 Page Modèle ML — métriques F1-score, précision, rappel et ROC-AUC 31
4.7 Page Rapports — statistiques par période, top attaquants et export CSV 32
4.8 Page Connexion — authentification et inscription avec gestion des sessions 32
4.9 Overlay mode vocal — orb animée, équaliseur et transcription en temps réel 33

5.1 Page Démo — pipeline IDS 5 étapes avec terminal live et carte GeoIP 35
5.2 Notification de blocage — countdown 30s, détails de l'attaque et validation humaine 35
5.3 Architecture globale de CyberSentinel-LLM 43

LIST OF TABLES

1.1 Comparaison IDS par signatures vs IDS comportemental 5
1.2 Comparaison Isolation Forest vs Local Outlier Factor 6

2.1 Exigences fonctionnelles de CyberSentinel-LLM 10

3.1 Description des composants backend 15

4.1 Features utilisées dans le pipeline ML 22
4.2 Échantillon de techniques MITRE ATT&CK indexées (697 au total dans ChromaDB) 23
4.3 Modes de défense du module de neutralisation 26
4.4 Endpoints principaux de l'API CyberSentinel-LLM 27

5.1 Métriques d'évaluation du module de détection 36
5.2 Synthèse des fonctionnalités — état final 37
5.3 Variables d'environnement principales 45

LISTE DES ABRÉVIATIONS

API : Application Programming Interface

ASN : Autonomous System Number

C2 : Command and Control

ChromaDB : Base de données vectorielle locale

DB : Database (Base de Données)

GeoIP : Geographic Internet Protocol

HTTP : HyperText Transfer Protocol

IA : Intelligence Artificielle

IDS : Intrusion Detection System

IF : Isolation Forest

IPS : Intrusion Prevention System

JSON : JavaScript Object Notation

JWT : JSON Web Token

LLM : Large Language Model

LOF : Local Outlier Factor

MITRE ATT&CK : Adversarial Tactics, Techniques and Common Knowledge

ML : Machine Learning

MVP : Minimum Viable Product

NER : Named Entity Recognition

ORM : Object Relational Mapping

PFE : Projet de Fin d'Études

RAG : Retrieval-Augmented Generation

RBAC : Role-Based Access Control

REST : Representational State Transfer

SIEM : Security Information and Event Management

SOC : Security Operations Center

SSH : Secure Shell

SSE : Server-Sent Events

STT : Speech-to-Text

TIC : Technologies de l'Information et de la Communication

TTS : Text-to-Speech

TTL : Time To Live

UI : User Interface

Vitest : Framework de tests JavaScript/TypeScript

INTRODUCTION GÉNÉRALE

Contexte et justification

Le Tchad s'engage depuis quelques années dans une transformation numérique de grande ampleur. En 2025, **2,74 millions d'internautes** représentent 13,2% des 20,8 millions d'habitants, tandis que **14,5 millions de connexions mobiles actives** attestent d'une connectivité en forte progression (69,8%). Sur le plan infrastructurel, plusieurs chantiers stratégiques avancent en parallèle : certification en cours du Data Center National, déploiement de liaisons en fibre optique vers le Niger, la Libye et l'Égypte, et cap fixé à 30% de pénétration Internet à l'horizon 2030.

Mais cette ouverture numérique expose simultanément le pays à des risques jusqu'alors peu ressentis. Le 10 janvier 2024, une attaque DDoS attribuée au groupe Anonymous Sudan a frappé **Airtel Tchad**, le premier opérateur télécom national, provoquant une interruption totale d'Internet à l'échelle du territoire. Parallèlement, les tentatives de brute-force SSH sur les serveurs Linux se multiplient dans la région, et les escroqueries visant les plateformes Mobile Money s'intensifient en Afrique centrale. Or, la réponse institutionnelle peine encore à suivre le rythme : les outils de type SIEM ou SOC nationaux restent rares, les audits de sécurité sont majoritairement confiés à des prestataires étrangers, et le vivier d'experts locaux demeure insuffisant — malgré des avancées récentes de l'ANSICE, notamment la formation de 30 cadres en cybersécurité en 2025, le lancement du Réseau national des RSSI en mai 2025, et l'audit de certification du Data Center National en février 2026.

C'est dans cet écart entre montée en charge numérique et fragilité défensive que s'inscrit **CyberSentinel-LLM** : une plateforme conçue pour automatiser la détection des comportements anormaux sur le réseau, assister l'analyste SOC par des explications contextuelles générées localement, et proposer des mesures de neutralisation soumises à validation humaine — sans dépendance à une infrastructure cloud.

Objectif général

L'ambition centrale de ce projet est de bâtir une plateforme cyberdéfensive intelligente et conversationnelle — **CyberSentinel-LLM** — qui articule trois briques complémentaires : un moteur de détection

d'anomalies fondé sur le Machine Learning non supervisé, un module de raisonnement contextuel s'appuyant sur le RAG et la base de connaissances MITRE ATT&CK, et un assistant local propulsé par un LLM capable de générer des analyses structurées en langage naturel. L'ensemble vise à rendre le triage des alertes SOC plus rapide, plus lisible et plus fiable, tout en maintenant l'analyste humain au centre de chaque décision défensive.

Objectifs spécifiques

- ☞ Développer un pipeline d'ingestion de logs et de détection d'anomalies (IsolationForest / LOF + fallback heuristique) ;
- ☞ Implémenter un module RAG indexant 697 techniques MITRE ATT&CK dans ChromaDB pour enrichir le raisonnement du LLM ;
- ☞ Intégrer un LLM local (Ollama) pour fournir des explications structurées en JSON avec score de confiance pondéré ;
- ☞ Implémenter un système de neutralisation adaptive : blocage applicatif middleware avec liste noire mémoire, notification countdown et approbation humaine obligatoire (Human-in-the-Loop) ;
- ☞ Implémenter une API backend (FastAPI) et une interface frontend (Next.js) avec streaming temps réel (SSE) et mode voix (STT/TTS) ;
- ☞ Fournir des outils de labellisation humaine et des métriques d'évaluation (precision, recall, F1, roc_auc) ;
- ☞ Documenter et déployer un scénario de démonstration reproductible localement.

Question centrale

Dans quelle mesure est-il possible de concevoir, sur du matériel ordinaire et sans dépendance cloud, un système de cyberdéfense conversationnel combinant détection par ML, enrichissement contextuel par RAG et génération d'explications par LLM, tout en garantissant une confiance mesurable, une neutralisation contrôlée par l'humain et une reproductibilité complète de l'architecture ?

Hypothèses

- ☞ **H1 :** Un pipeline minimaliste basé sur des features orientées SSH permet de démontrer un fil rouge pédagogique complet et cohérent ;
- ☞ **H2 :** Les algorithmes d'anomalie standards (IsolationForest, LOF) suffisent pour un MVP pédagogique combinés à un fallback heuristique robuste ;

- ☞ **H3 :** Un LLM local enrichi par RAG MITRE ATT&CK améliore significativement l'explicabilité et la précision des analyses d'incidents ;
- ☞ **H4 :** Un système de blocage applicatif avec validation humaine (Human-in-the-Loop) constitue une forme viable de neutralisation adaptive dans un contexte pédagogique.

Positionnement du projet

CyberSentinel-LLM se revendique avant tout comme une **plateforme cognitive de cybersécurité à vocation pédagogique** : son objectif est de prouver, à l'échelle d'un Projet de Fin d'Études de Licence, qu'un SOC augmenté par LLM et doté d'une capacité de neutralisation est techniquement réalisable sur un périmètre ciblé. Ce projet n'ambitionne pas de concurrencer des solutions industrielles, mais d'établir les fondements conceptuels et techniques d'une telle approche, ancrée dans les contraintes et les besoins réels du contexte tchadien.

CHAPTER 1

GÉNÉRALITÉS

Introduction

Protéger une infrastructure numérique suppose d'abord de comprendre les outils disponibles pour détecter et contenir les intrusions. Ce chapitre pose les bases conceptuelles du projet en parcourant l'évolution des systèmes IDS/IPS, en exposant les lacunes que les approches traditionnelles laissent ouvertes, puis en introduisant les apports du Machine Learning, des grands modèles de langage et du RAG pour y répondre de façon complémentaire.

1.1 Présentation des IDS et IPS

Un **IDS (Intrusion Detection System)** surveille en continu le trafic réseau ou les journaux d'événements pour y repérer des comportements suspects. Son cousin l'**IPS (Intrusion Prevention System)** va un cran plus loin en bloquant de façon proactive les flux jugés malveillants. Deux grandes familles de méthodes structurent ce domaine :

- ☞ **Détection par signatures :** chaque trafic entrant est confronté à un répertoire de signatures d'attaques répertoriées. Des solutions comme *Snort* ou *Suricata* incarnent cette approche : redoutablement efficaces face aux menaces connues, elles restent néanmoins aveugles dès qu'une attaque sort des patterns enregistrés (attaques zero-day) ;
- ☞ **Détection comportementale :** plutôt que de chercher une signature, ces systèmes modélisent le comportement habituel du réseau et lèvent une alerte à chaque écart significatif. Ils offrent une meilleure couverture sur les menaces inédites, mais génèrent davantage de faux positifs, ce qui peut surcharger les équipes d'analyse.

Table 1.1: Comparaison IDS par signatures vs IDS comportemental

Critère	**Par signatures**	**Comportemental**
Détection menaces connues	Excellente	Moyenne
Détection zero-day	Nulle	Bonne
Taux de faux positifs	Faible	Élevé
Mise à jour requise	Fréquente (base signatures)	Non
Exemples d'outils	Snort, Suricata	IsolationForest, LOF

1.2 Limites des approches classiques

Les solutions IDS/IPS conventionnelles se heurtent à quatre obstacles majeurs dans les environnements opérationnels actuels :

- ☞ **Inflation des alertes :** un SOC actif peut traiter plusieurs milliers de notifications par jour. Sans outillage intelligent pour les hiérarchiser, l'analyste se retrouve rapidement débordé et incapable d'identifier ce qui mérite vraiment son attention ;
- ☞ **Faux positifs chroniques :** une fraction significative des alertes se révèle inoffensive après examen. Cette pollution quotidienne érode la vigilance des équipes et risque de faire passer à la trappe des événements critiques noyés dans la masse ;
- ☞ **Absence d'explication :** un IDS classique signale une anomalie, mais ne dit pas pourquoi elle est dangereuse ni comment y répondre. L'analyste doit reconstruire seul le contexte de l'incident, un exercice long et exigeant ;
- ☞ **Isolement des événements :** les alertes sont traitées individuellement, sans mécanisme natif pour les relier entre elles et reconstituer une chaîne d'attaque cohérente — une limite dommageable face aux campagnes adversariales multi-étapes.

1.3 Apport du Machine Learning en détection d'anomalies

Sous-domaine de l'Intelligence Artificielle, le Machine Learning donne à un programme la capacité d'identifier des régularités dans les données sans qu'on lui dicte explicitement les règles à appliquer. Appliqué à la cybersécurité, il permet de modéliser ce qu'est un trafic normal, puis de signaler tout comportement qui s'en éloigne. Deux algorithmes non supervisés se distinguent particulièrement pour cette tâche :

- ☞ **Isolation Forest (IF) :** cet algorithme construit aléatoirement des arbres de décision et mesure le nombre de coupures nécessaires pour isoler chaque observation. Une anomalie, de par sa rareté

et son éloignement des données habituelles, est isolée en très peu d'étapes. Le score brut est ensuite normalisé via une sigmoïde pour obtenir une valeur bornée entre 0 et 1 [1] ;

- ☞ **Local Outlier Factor (LOF) :** cet algorithme évalue la densité du voisinage immédiat de chaque point et la compare à celle de ses voisins. Un point entouré d'une zone peu dense alors que ses voisins évoluent dans un espace compact est classé comme outlier [2].

Ces méthodes comportent leurs propres contraintes : elles sont sensibles à la sélection des variables d'entrée, peuvent souffrir d'un glissement de domaine si les données d'entraînement ne représentent pas bien le trafic courant, et nécessitent un historique suffisant pour calibrer le modèle.

Table 1.2: Comparaison Isolation Forest vs Local Outlier Factor

Critère	**Isolation Forest**	**Local Outlier Factor**
Type	Ensemble (arbres)	Densité locale
Complexité	Faible	Moyenne
Données requises	Peu	Moyenne
Interprétabilité	Faible	Faible
Sensibilité au bruit	Robuste	Sensible
Usage dans le projet	Score principal	Score secondaire

1.4 Apport des LLM et principe Human-in-the-Loop

L'émergence des **Grands Modèles de Langage (LLM)** ouvre un nouveau registre d'assistance pour les analystes SOC [4]. Là où un IDS se contente de signaler, un LLM peut aller beaucoup plus loin :

- ☞ **Traduire l'alerte en langage humain :** plutôt qu'un code d'erreur ou une signature brute, l'analyste reçoit une description intelligible de ce qui s'est passé et de ce que cela implique ;
- ☞ **Tisser des liens entre événements :** le LLM peut regrouper plusieurs alertes corrélées et reconstituer le déroulé probable d'une attaque, aidant ainsi à transformer un bruit diffus en scénario cohérent ;
- ☞ **Suggérer des pistes d'action :** le modèle formule des recommandations de remédiation, sans jamais les exécuter lui-même — la décision reste entièrement entre les mains de l'analyste [5].

Ce dernier point illustre le principe fondateur du **Human-in-the-Loop** : le LLM est un outil d'aide à la décision, jamais un acteur autonome sur le système. Cette philosophie protège contre les actions défensives erronées déclenchées par une analyse incomplète ou une hallucination du modèle.

Dans **CyberSentinel-LLM**, ce rôle est tenu par le modèle *qwen2.5:1.5b* déployé via **Ollama** en local, ce qui garantit la confidentialité des logs réseau traités et s'affranchit de toute dépendance à une API commerciale externe.

1.5 RAG pour la cybersécurité

La méthode **RAG (Retrieval-Augmented Generation)**, formalisée par Lewis et al. [3], consiste à brancher un LLM sur une base de connaissances externe : avant de générer une réponse, le modèle récupère dynamiquement les fragments les plus pertinents et les intègre dans son raisonnement.

Appliqué à la cybersécurité, ce mécanisme apporte trois bénéfices concrets :

- ☞ **Réponses ancrées dans des faits vérifiables :** plutôt que de puiser uniquement dans ses données d'entraînement générales, le LLM s'appuie sur des références spécialisées et actualisées, comme MITRE ATT&CK ou des bulletins CVE, ce qui réduit significativement le risque d'hallucination ;
- ☞ **Enrichissement incrémental de la base :** ajouter de nouvelles techniques d'attaque ou de nouveaux indicateurs ne nécessite pas de réentraîner le modèle — il suffit de mettre à jour la collection vectorielle ;
- ☞ **Traçabilité des analyses :** chaque explication produite peut être rattachée à une technique MITRE précise, ce qui permet à l'analyste de vérifier le raisonnement du système et d'en évaluer la pertinence.

Le référentiel **MITRE ATT&CK** (version Enterprise 18) recense **216 techniques et 475 sous-techniques** [7], documentant pour chacune les tactiques adversariales, les indicateurs de détection et les mesures d'atténuation recommandées. C'est la base de connaissances la plus complète et la plus utilisée par la communauté cybersécurité mondiale.

Dans **CyberSentinel-LLM**, **697 entrées MITRE ATT&CK Enterprise** — parsées depuis le bundle STIX officiel via `scripts/fetch_mitre.py` — sont vectorisées avec le modèle d'embedding *all-MiniLM-L6-v2* (384 dimensions, CPU uniquement) et stockées localement dans ChromaDB, rendant la recherche sémantique entièrement autonome.

Conclusion

Ce tour d'horizon a permis de mesurer les apports mais aussi les fragilités des systèmes IDS/IPS conventionnels face aux cybermenaces actuelles. Il a également mis en lumière comment le Machine Learning, les LLM et le RAG peuvent combler ces lacunes de façon complémentaire. Ces fondements théoriques justifient pleinement les choix architecturaux de **CyberSentinel-LLM** : ML pour la détection précoce des anomalies, RAG MITRE ATT&CK pour l'enrichissement contextuel, et validation humaine systématique avant toute action défensive. Le chapitre suivant traduit ces orientations en exigences concrètes et en cas d'usage opérationnels.

CHAPTER 2

ANALYSE DES BESOINS ET SPÉCIFICATIONS

Introduction

Avant d'écrire la première ligne de code, il faut savoir précisément ce que le système doit accomplir et dans quelles conditions il doit fonctionner. Ce chapitre synthétise ce travail d'analyse : face à un contexte tchadien caractérisé par un outillage SOC embryonnaire, une dépendance aux solutions étrangères et des contraintes matérielles réelles, **CyberSentinel-LLM** doit répondre à des exigences fonctionnelles et non fonctionnelles clairement articulées, tout en restant intégralement déployable sur une machine ordinaire.

2.1 Cas d'usage principaux

Les fonctionnalités de **CyberSentinel-LLM** ont été définies en partant du quotidien concret d'un analyste SOC qui doit gérer des attaques réseau avec des ressources limitées. Sept grandes interactions ont été identifiées :

- ☞ **Détection automatique d'anomalies :** ingestion de logs réseau (SSH, authentification), scoring ML et génération automatique d'alertes avec niveau de sévérité ;
- ☞ **Triage et gestion des incidents :** regroupement d'alertes corrélées en incidents, suivi du statut et de la confiance ;
- ☞ **Explication d'incidents via LLM + RAG :** génération automatique d'une analyse contextuelle en langage naturel enrichie de techniques MITRE ATT&CK pour chaque incident ;
- ☞ **Neutralisation adaptive :** notification urgente avec countdown, approbation humaine du blocage applicatif et génération des commandes iptables correspondantes ;
- ☞ **Tableau de bord SOC :** visualisation des KPI, timeline des alertes et cartographie GeoIP des sources d'attaque ;

☞ **Interface conversationnelle :** chat en langage naturel avec le LLM en mode texte et voix (STT/TTS locaux) ;

☞ **Labellisation et métriques de calibration :** validation humaine des alertes (`true_attack` / `false_positive`) et calcul de métriques (precision, recall, F1, roc_auc).

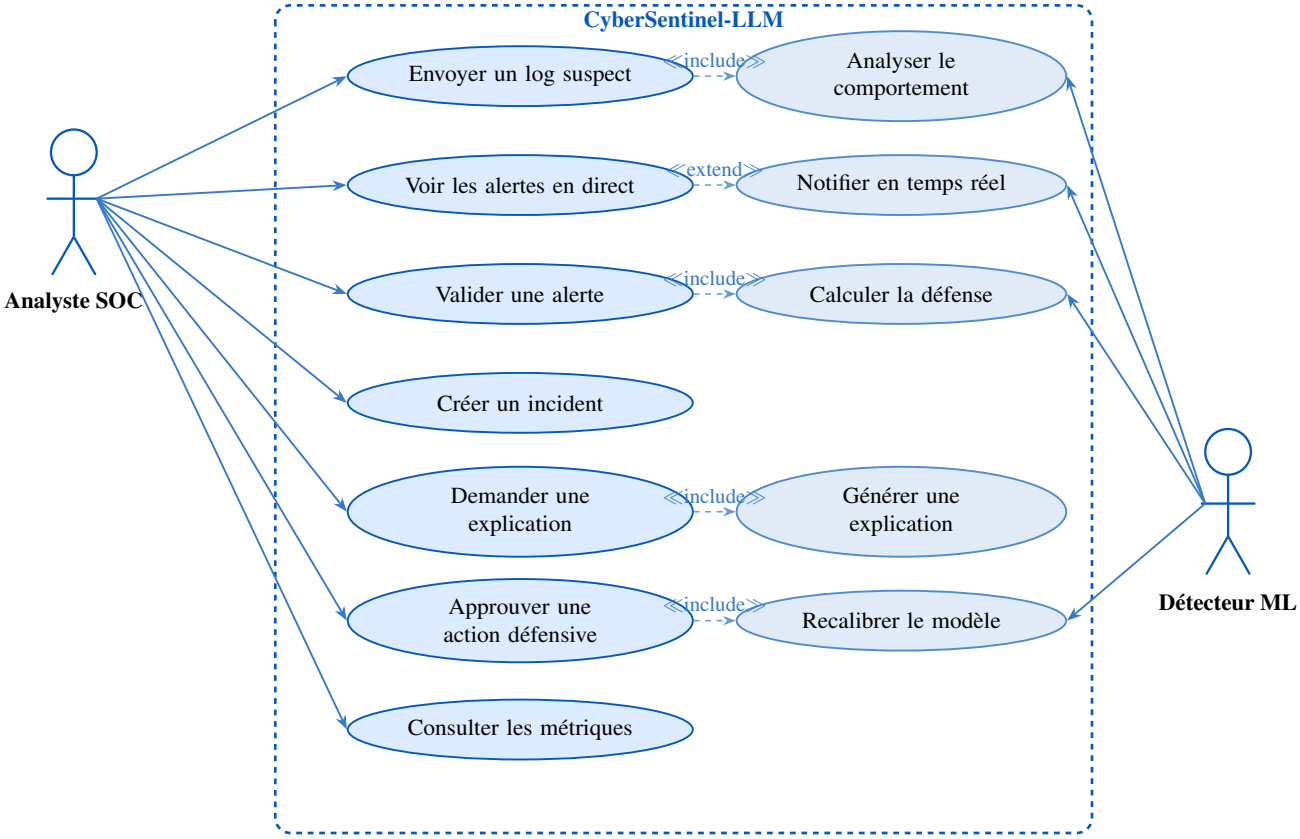


Figure 2.1: Diagramme de cas d'utilisation de CyberSentinel-LLM

2.2 Exigences fonctionnelles

Treize exigences fonctionnelles ont été formalisées (EF01 à EF13) et rassemblées dans le Tableau 2.1. Elles couvrent l'intégralité du cycle de traitement d'un incident : depuis l'ingestion et le scoring des logs réseau jusqu'à la neutralisation des menaces, en passant par la gestion des alertes et des incidents, la recherche sémantique dans MITRE ATT&CK, la génération d'explications structurées par le LLM, le streaming temps réel et l'authentification à double facteur.

Table 2.1: Exigences fonctionnelles de CyberSentinel-LLM

ID	Fonctionnalité	Description
EF01	Ingestion de logs	Recevoir et traiter 1..N logs via POST /ingest/logs
EF02	Scoring ML	Calculer un score d'anomalie 0..1 par log ingéré (IF/LOF/hybride)
EF03	Gestion des alertes	Lister, filtrer et labelliser les alertes générées
EF04	Gestion des incidents	Créer, consulter et suivre les incidents
EF05	RAG MITRE ATT&CK	Indexer 697 techniques et enrichir le contexte LLM
EF06	Explication LLM	Générer une explication JSON structurée d'un incident
EF07	Score de confiance	Calculer confidence = 0.4×ML + 0.3×RAG + 0.3×LLM
EF08	Neutralisation adaptive	Blocage applicatif middleware avec liste noire mémoire
EF09	Notification blocage	Afficher countdown 30s et demander approbation humaine
EF10	Chat conversationnel	Interface de chat avec le LLM local (texte + voix)
EF11	Streaming temps réel	Diffuser les alertes en temps réel via SSE
EF12	Cartographie GeoIP	Afficher la localisation géographique des IPs sources
EF13	Métriques ML	Calculer et afficher precision/recall/F1/roc_auc

2.3 Exigences non fonctionnelles

Au-delà des fonctionnalités, la plateforme doit respecter des contraintes de qualité, de performance et de sécurité qui conditionnent sa viabilité dans le contexte tchadien :

- ☞ **Réplicabilité locale :** le système doit être entièrement déployable sur une machine locale sans dépendance à des services cloud (Ollama local, SQLite, ChromaDB local) ;
- ☞ **Compatibilité matérielle contrainte :** toute la chaîne de traitement (ML + embedding RAG + LLM) doit fonctionner sur CPU Intel i7 11ème génération avec 16 GB de RAM, sans GPU ;
- ☞ **Performance minimale :** le scoring ML doit s'exécuter en moins de 50 ms par alerte ; l'explication LLM est tolérée entre 30 et 90 secondes sur CPU ;
- ☞ **Sécurité minimale :** CORS configurable, rate limiting par IP, security headers HTTP, limitation de taille des requêtes ;
- ☞ **Maintenabilité :** architecture modulaire (routers / services / models) facilitant l'extension et la maintenance ;

☞ **Principe Human-in-the-Loop :** le LLM ne prend aucune décision autonome sur le système ; toute action défensive de type blocage requiert une approbation humaine explicite.

Conclusion

Ce chapitre a établi le référentiel des besoins de **CyberSentinel-LLM** : les cas d'usage qui reflètent la réalité d'un analyste SOC, les treize exigences fonctionnelles qui en découlent, et les contraintes non fonctionnelles qui borneront les choix d'implémentation. Ce cadre servira de boussole pour la conception architecturale et le développement détaillés dans les chapitres qui suivent.

CHAPTER 3

CONCEPTION LOGICIELLE ET ARCHITECTURE

Introduction

Après avoir défini ce que le système doit faire, il faut maintenant décrire comment il est construit. Ce chapitre expose l'architecture globale de **CyberSentinel-LLM**, l'organisation du code source, le rôle précis de chaque composant backend et frontend, ainsi que les diagrammes UML qui modélisent les données persistées et les flux d'interaction entre modules.

3.1 Architecture globale

Le système est structuré en une **architecture monolithique modulaire orientée API, organisée en cinq couches superposées**. Ce parti pris architectural simplifie le déploiement sur une machine locale unique tout en maintenant une séparation nette des responsabilités — chaque couche ayant un périmètre fonctionnel bien délimité et ne communiquant avec les autres que par des interfaces définies.

Ces cinq couches sont les suivantes :

- **Couche 1 — Frontend (UI) :** interface Next.js 16 (App Router), 9 pages SOC (dashboard, alertes, incidents, demo, chat, modèle ML, paramètres, rapports, login), intégration SSE temps réel ;
- **Couche 2 — API Gateway :** FastAPI modulaire avec routers, endpoints, middlewares de sécurité et middleware de blocage IP ;
- **Couche 3 — Détection ML :** pipeline non supervisé + heuristique (IsolationForest, LOF, fallback), scoring 0..1, règles de sévérité ;
- **Couche 4 — RAG + LLM :** ChromaDB avec 697 techniques MITRE ATT&CK (bundle STIX Enterprise), embedding all-MiniLM-L6-v2, Ollama pour explication structurée et chat conversationnel ;

☞ **Couche 5 — Défense :** moteur de décision défensive, liste noire mémoire O(1), middleware blocage IP (HTTP 403), génération commandes iptables.

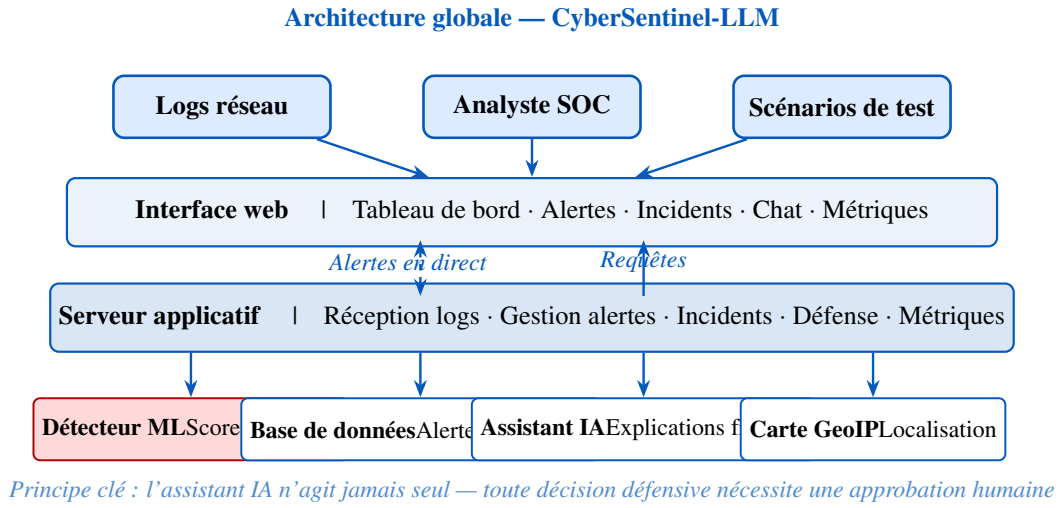


Figure 3.1: Architecture globale de CyberSentinel-LLM

3.2 Arborescence du projet

Le dépôt du projet reflète fidèlement cette organisation en couches. Voici l'arborescence complète :

Structure du projet CyberSentinel-LLM

```
cybersentinel-llm/
|-- backend/
|   |-- app/
|   |   |-- core/           # Configuration et settings
|   |   |-- db/             # Connexion base de donnees
|   |   |-- routers/        # Contrats API metier
|   |   |-- endpoints/      # Endpoints LLM avances (chat, TTS, STT)
|   |   |-- models/         # Entites SQLModel (8 tables)
|   |   `-- services/       # Logique technique transverse
|   |       |-- detection.py    # ML : IF, LOF, heuristique
|   |       |-- rag_service.py  # ChromaDB + MITRE ATT&CK
|   |       |-- llm_ollama.py   # Ollama + RAG + confiance
|   |       |-- ip_blacklist.py # Liste noire memoire
|   |       |-- defense.py      # Moteur decision defensive
|   |       |-- geoip.py        # Lookup MaxMind GeoLite2
|   |       |-- sse.py          # Server-Sent Events
|   |       |-- tts_service.py  # Synthese vocale Piper
|   |       `-- whisper_service.py  # Transcription Whisper
|-- frontend/
|   |-- app/
|   |   |-- dashboard/      # Vue temps reel
|   |   |-- alerts/         # Gestion alertes + blocage
|   |   |-- incidents/      # Incidents + explication LLM
|   |   |-- demo/           # Pipeline 5 etapes anime
|   |   |-- chat/           # Chat SOC texte + voix
|   |   |-- model/          # Metriques ML
|   |   |-- settings/       # Parametres + IPs bloquees
|   |   |-- reports/        # Rapports + export CSV
|   |   `-- login/          # Authentification
|   `-- components/
|       |-- chat/
|       |   |-- types.ts             # Types partages (Conversation, ChatMsg)
|       |   |-- ConversationSidebar.tsx  # Sidebar conversations
|       |   |-- MessageRenderer.tsx  # Rendu messages + helpers
|       |   |-- OverviewCards.tsx    # Cartes alertes ML
|       |   `-- VoiceModeOverlay.tsx # Overlay mode vocal
|       |-- settings/
|       |   `-- shared.tsx           # StatusChip, ServiceRow, CopyBtn...
|       |-- BlockingNotification.tsx  # Notification blocage
|       |-- WorldMap.tsx              # Carte GeoIP Leaflet
|       `-- ...
`-- chroma_db/              # Persistance ChromaDB locale
```

3.3 Description des composants backend

Chaque fichier du répertoire `services/` encapsule une responsabilité technique précise et indépendante. Le tableau suivant en détaille le rôle :

Table 3.1: Description des composants backend

Composant	Rôle
`services/detection.py`	Extraction de 5 features, entraînement IF/LOF, scoring 0..1, fallback heuristique
`services/rag_service.py`	ChromaDB, embedding all-MiniLM-L6-v2, 697 techniques MITRE ATT&CK (STIX Enterprise), retrieve_context()
`services/llm_ollama.py`	Prompt RAG enrichi, appel Ollama, parsing JSON, retry, confiance pondérée
`services/ip_blacklist.py`	Liste noire mémoire thread-safe, synchronisation DB, boucle d'expiration
`services/defense.py`	Moteur de décision défensive (4 modes), TTL blocage
`services/geoip.py`	Lookup MaxMind GeoLite2-City.mmdb
`services/sse.py`	Queues asyncio multi-abonnés, publication alertes temps réel
`services/tts_service.py`	Synthèse vocale Piper locale
`services/whisper_service.py`	Transcription vocale STT locale
`routers/ingest.py`	Ingestion logs, scoring ML, GeoIP, SSE
`routers/defense.py`	Endpoints approve/revoke/block-manual
`routers/metrics.py`	Calcul precision/recall/F1/roc_auc
`models/`	Alert, Incident, LLMTrace, IPEnrichment, AlertLabel, Conversation, ChatMessage, DefenseAction

3.4 Description du frontend

L'interface utilisateur est construite avec **Next.js 16** (App Router) en thème sombre adapté au contexte SOC. Elle expose neuf pages fonctionnelles :

- ☞ **/dashboard :** vue temps réel avec 4 KPIs, timeline 24h, distribution sévérité, carte GeoIP, 10 dernières alertes, badge LIVE ;
- ☞ **/alerts :** liste complète des alertes, labellisation (true_attack / false_positive), approbation/révocation des blocages ;
- ☞ **/incidents/[id] :** détail d'un incident, explication LLM enrichie RAG, techniques MITRE identifiées, décision humaine ;
- ☞ **/demo :** pipeline IDS 5 étapes animé avec terminal live, carte GeoIP, explication LLM et blocage automatisé ;
- ☞ **/chat :** console conversationnelle streaming avec historique persisté, mode voix (STT Whisper + TTS Piper) ;

- ☞ **/model :** métriques ML (F1-score, précision, rappel, ROC-AUC, seuil actif) ;
- ☞ **/settings :** état des services (Ollama, SQLite, RAG, GeoLite2), configuration du mode défense, gestion des IPs bloquées (liste, blocage manuel, commandes iptables), authentification 2FA TOTP (QR code, activation/désactivation), gestion des utilisateurs (CRUD) et administration système (réinitialisation de la base de données) ;
- ☞ **/reports :** statistiques par période, top attaquants, distribution MITRE, export CSV/TXT ;
- ☞ **/login :** authentification et inscription.

3.5 Refactoring frontend — Architecture modulaire

À mesure que le frontend grossissait, certains fichiers ont dépassé le millier de lignes, rendant la navigation et les tests difficiles. Une refactorisation ciblée a été conduite pour extraire les sous-composants réutilisables et les types partagés, sans modifier le comportement observable de l'application.

Refactoring de la page Chat

- ☞ **Fichier créé :** `components/chat/types.ts` — Types partagés (`Conversation`, `ChatMsg`, `OVERVIEW_MARKER`) ;
- ☞ **Composant OverviewCards :** affichage des cartes alertes ML détectées dans les réponses LLM ;
- ☞ **Composant ConversationSidebar :** sidebar de gestion des conversations (créer, renommer, supprimer) ;
- ☞ **Composant MessageRenderer :** rendu des messages avec helpers (Linkified, InlineCode, CodeBlock) et support TTS ;
- ☞ **Composant VoiceModeOverlay :** overlay plein écran pour le mode vocal avec animations CSS (orb breathing, ripple rings, equalizer).

Résultat : réduction de `chat/page.tsx` de 1395 à 884 lignes (-37%), avec une séparation claire des responsabilités.

Refactoring de la page Settings

- ☞ **Fichier créé :** `components/settings/shared.tsx` — Composants partagés (`StatusChip`, `ServiceRow`, `CopyBtn`, `ExpiryCountdown`) et utilitaires (`timeLeft`) ;
- ☞ **Composants extraits :**

- `ServiceStatusSection` — État des services (Ollama, SQLite, RAG, GeoLite2) ;
- `DefenseConfigSection` — Configuration défense + mode actif ;
- `BlockedIpsSection` — Liste IPs bloquées + blocage manuel ;
- `MfaSection` — Configuration 2FA (QR code, validation) ;
- `UsersSection` — Gestion utilisateurs (CRUD) ;
- `SystemAdminSection` — Administration système (reset DB).

Nettoyage du code mort

☞ **Fichier supprimé :** `lib/llm.ts` — Non utilisé (remplacé par appel direct Ollama) ;

☞ **Fonctions supprimées dans utils.ts :** `formatTimestamp()`, `severityColor()`, `severityIcon(` — Jamais utilisées hors fichier ;

☞ **Consolidation localStorage :** constantes dupliquées dans `lib/actor.ts` remplacées par imports depuis `AuthProvider`.

3.6 Diagrammes UML

Trois diagrammes UML complètent la description architecturale : le diagramme de classes capture le schéma de persistance et les relations entre entités, le diagramme de séquence retrace le cycle de vie complet d'une alerte depuis son arrivée jusqu'à l'explication fournie à l'analyste, et le diagramme d'activité illustre la logique de branchement du pipeline de décision.

3.6.1 Diagramme de classes

La Figure 3.2 présente les entités persistées en base de données et leurs relations. L'entité `Alert` centralise les événements détectés et peut être regroupée en `Incident`, chacun disposant d'une trace d'explication LLM (`LLMTrace`). Les actions défensives (`DefenseAction`) et les labels de validation humaine (`AlertLabel`) sont rattachés aux alertes, tandis que les conversations du chat SOC (`Conversation`, `ChatMessage`) forment un sous-graphe indépendant.

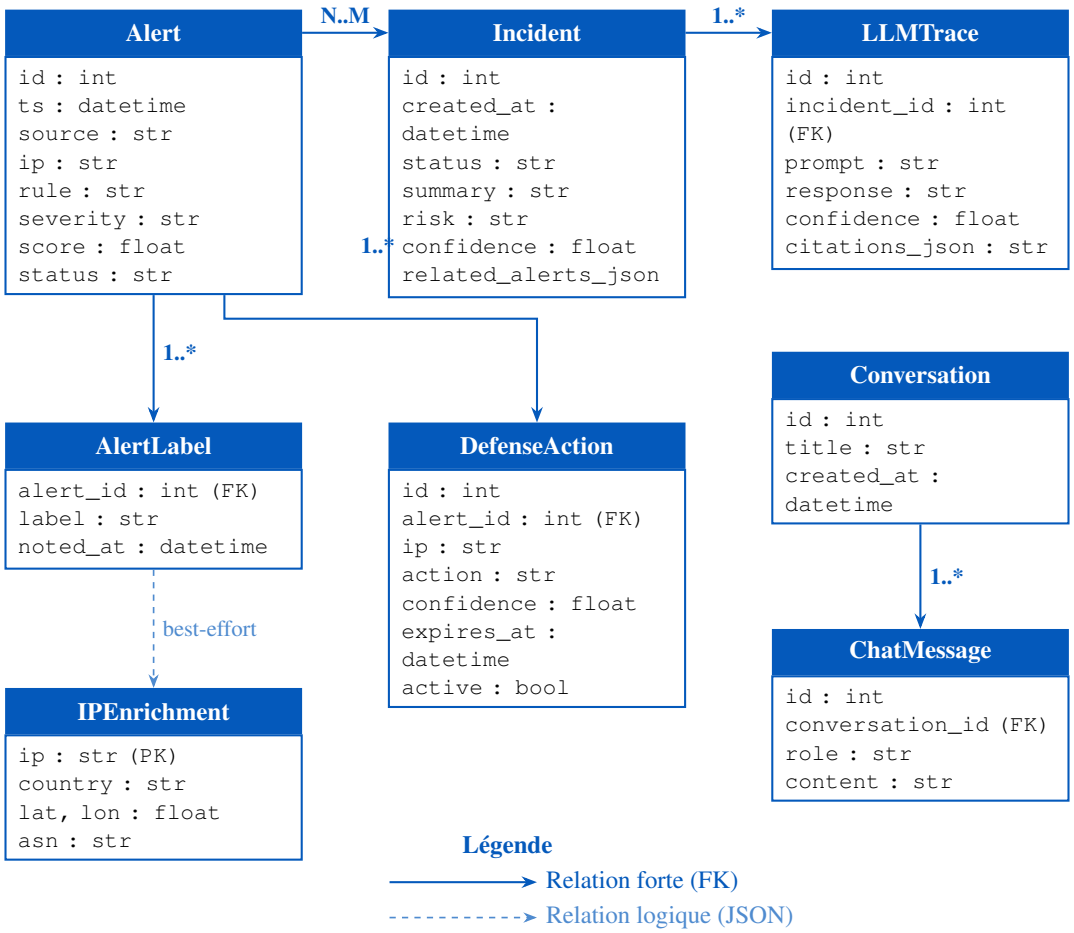


Figure 3.2: Diagramme de classes de CyberSentinel-LLM

3.6.2 Diagramme de séquence

La Figure 3.3 illustre le flux complet d'une alerte depuis l'arrivée d'un log suspect jusqu'à l'affichage de l'explication structurée à l'analyste SOC. Le log est d'abord soumis au pipeline ML qui calcule un score d'anomalie et enregistre l'alerte en base. Le moteur de défense détermine alors l'action appropriée (monitor, review ou block_temp) et déclenche une notification temps réel via SSE. L'analyste consulte l'alerte, approuve ou rejette l'action défensive, ce qui provoque la création d'un incident et l'appel au module RAG + LLM pour générer l'explication contextuelle finale.

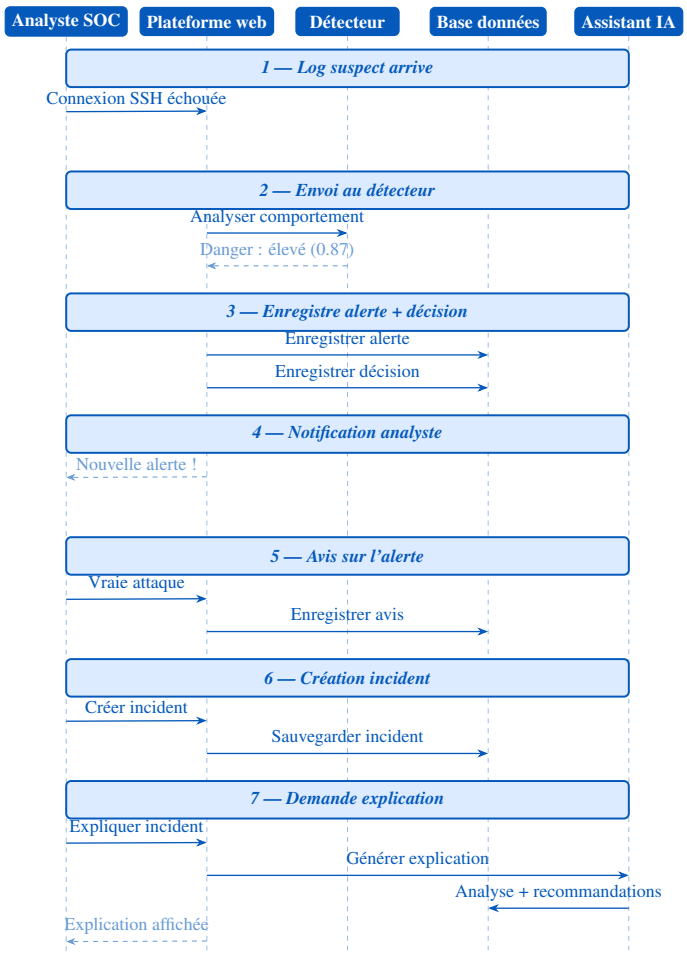


Figure 3.3: Diagramme de séquence — flux d'une alerte jusqu'à la neutralisation

3.6.3 Diagramme d'activité

La Figure 3.4 décrit le processus de traitement d'un log entrant sous forme d'activités séquentielles et de branchements conditionnels. Après l'analyse comportementale, le score d'anomalie oriente le flux : un score faible conduit à l'archivage du log sans alerte, tandis qu'un score élevé déclenche la classification à haute sévérité, l'enregistrement de l'alerte et la notification de l'analyste. L'analyste valide ou rejette l'alerte, ce qui détermine la création ou non d'un incident ; dans les deux cas, son retour (`true_attack` / `false_positive`) est utilisé pour calibrer le détecteur ML.

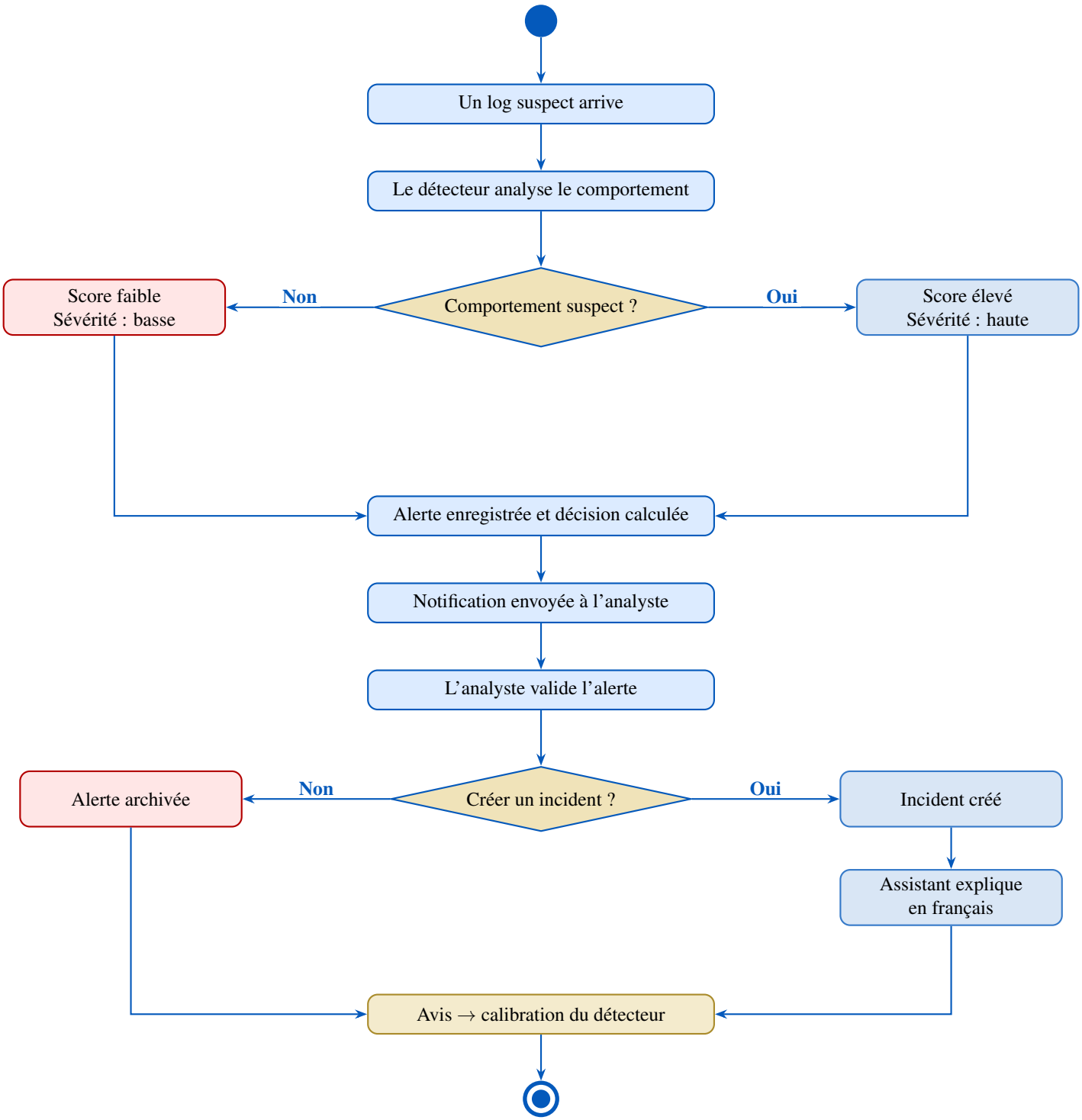


Figure 3.4: Diagramme d'activité — traitement d'un log entrant

Conclusion

Ce chapitre a dressé un portrait complet de **CyberSentinel-LLM** vu de l'intérieur : son architecture en cinq couches, l'organisation de son code, le rôle de chacun de ses composants et les flux d'interaction modélisés en UML. La conception étant clairement posée, le chapitre suivant plonge dans les détails d'implémentation de chaque brique technique.

CHAPTER 4

IMPLÉMENTATION

Introduction

Ce chapitre ouvre le capot du système et documente les choix d'implémentation concrets : comment le pipeline ML calcule ses scores, comment le module RAG interroge la base MITRE ATT&CK, comment le LLM construit ses analyses avec un score de confiance pondéré, comment la neutralisation s'enclenche sans jamais court-circuiter l'analyste, et comment le flux temps réel par SSE relie toutes ces briques à l'interface web.

4.1 Pipeline Machine Learning

Chaque log réseau ingéré traverse cinq étapes successives avant de déboucher sur une alerte classifiée :

1. **Extraction des features** depuis le log brut (5 features) ;
2. **Construction de la matrice d'entraînement** : `_build_training_matrix(N=200)` à partir des 200 dernières alertes ;
3. **Scoring** via IF/LOF/hybride selon la variable d'environnement `DETECTOR` ;
4. **Application du seuil** `SCORE_THRESHOLD` (défaut : 0.6) ;
5. **Mapping score $\rightarrow$ sévérité** : high (≥ 0.8), medium (≥ 0.5), low (< 0.5).

Table 4.1: Features utilisées dans le pipeline ML

Feature	Description / Formule
`is_ssh`	1.0 si source == "ssh", sinon 0.0
`is_fail`	1.0 si status == "fail", sinon 0.0
`hour_sin`	$\sin(2\pi \times (h + m/60)/24)$
`hour_cos`	$\cos(2\pi \times (h + m/60)/24)$
`ip_fail_count_60m`	Nombre d'alertes fail pour cette IP sur 60 min (SQLite)

Scoring IsolationForest : le score décisionnel brut est transformé via une fonction sigmoïde inverse :

Formule IsolationForest

$$P_{normal} = \frac{1}{1 + e^{-k \cdot df}}, \quad k = 4 \qquad score_{IF} = 1 - P_{normal} \in [0, 1]$$

Scoring LOF (mode novelty) : le modèle LOF est entraîné une seule fois sur la matrice d'historique (`novelty=True`). Le score d'un nouveau point est obtenu via `score_samples()`, puis normalisé :

Formule LOF — normalisation novelty

$$score_{LOF} = \max\left(0,\ \min\left(1,\ \frac{\max(0,\ -s(x) - 1)}{3}\right)\right)$$

Où $s(x) = \texttt{score_samples}(x) \leq 0$; $s(x) \approx -1$ pour un inlier (score $\to 0$), $s(x) \ll -1$ pour une anomalie (score $\to 1$).

Mode hybride : $score = \max(score_{IF}, score_{LOF})$

Fallback heuristique (< 20 échantillons en base) :

Formule de fallback heuristique

$$score = 0.2 + 0.5 \times is_fail + \min(ip_fail_count, 5) \times 0.05$$

4.2 Module RAG — MITRE ATT&CK

Le module RAG est la pièce qui transforme un LLM généraliste en un assistant cybersécurité crédible : au lieu de se fier uniquement à ce qu'il a appris lors de son entraînement, le modèle consulte dynamiquement une base de connaissances spécialisée avant de formuler son analyse.

Architecture ChromaDB

- ☞ **Collection :** `mitre_attack`, distance cosinus (`hnsw:space: cosine`) ;
- ☞ **Persistance :** dossier `chroma_db/` à la racine du projet ;
- ☞ **Initialisation idempotente :** ne réindexe que si la collection contient moins de techniques que le fichier JSON source (`json_count > collection_count`) — aucune pénalité au redémarrage ;
- ☞ **Modèle d'embedding :** *all-MiniLM-L6-v2* (sentence-transformers, ∼80 MB, CPU-only, 384 dimensions).

Techniques MITRE ATT&CK indexées

La base ChromaDB contient **697 techniques** parsées depuis le bundle STIX officiel MITRE ATT&CK Enterprise. Le tableau ci-dessous présente un échantillon représentatif des techniques les plus pertinentes pour le scénario SSH de démonstration :

Table 4.2: Échantillon de techniques MITRE ATT&CK indexées (697 au total dans ChromaDB)

ID	Nom	Sévérité
T1110.001	Password Guessing (Brute Force SSH)	high
T1110.003	Password Spraying	high
T1078	Valid Accounts	high
T1021.004	SSH Remote Services	medium
T1059.004	Unix Shell	high
T1190	Exploit Public-Facing Application	high
T1046	Network Service Discovery	medium
T1595.001	Scanning IP Blocks	medium
T1133	External Remote Services	medium
T1071.001	Web Protocols (C2)	high
T1048	Exfiltration Over Alternative Protocol	high
T1098	Account Manipulation	high
T1136	Create Account	medium
T1070.003	Clear Command History	medium
T1562.001	Disable Security Tools	high

Score de pertinence RAG : la similarité cosinus est calculée depuis la distance ChromaDB :

Score de pertinence RAG

$$similarity_{RAG} = \max\left(0,\ \min\left(1,\ 1 - \frac{distance}{2}\right)\right) \quad \in [0, 1]$$

4.3 Module LLM et Score de Confiance

C'est le module LLM qui concrétise la promesse centrale de **CyberSentinel-LLM** : transformer une alerte brute en une explication structurée, intelligible et traçable, grâce au contexte injecté par le RAG.

Modèle utilisé

Modèle : *qwen2.5:1.5b* via Ollama local (`http://localhost:11434`), avec sortie JSON forcée (`"format": "json"`).

Optimisations pour inférence CPU : afin de garantir des temps de réponse acceptables (30–90 secondes sur CPU sans GPU), plusieurs paramètres ont été configurés :

- ☞ `keep_alive: -1` — maintient le modèle chargé en RAM entre les requêtes, évitant le rechargement coûteux à chaque appel ;
- ☞ `num_predict: 600` — limite la longueur de la réponse générée pour réduire le temps d'inférence ;
- ☞ `num_ctx: 2048` — fenêtre de contexte adaptée aux prompts RAG enrichis ;
- ☞ `temperature: 0.1` — sortie déterministe favorisant des réponses JSON stables et parsables ;
- ☞ **Warm-up au démarrage** — un appel initial au démarrage du serveur précharge le modèle en RAM.

Prompt template RAG enrichi

Template de prompt RAG — Explication d'incident

```
Tu es un expert SOC spécialisé en cybersécurité.

Contexte MITRE ATT&CK récupéré automatiquement :
{rag_context}

Incident à analyser : {incident_summary}
Alertes associées : {alert_ids}

Réponds UNIQUEMENT avec un objet JSON valide :
{
  "severity_score": <entier 0-10>,
  "attack_type": "<nom court en français>",
  "mitre_technique": "<ID ex: T1110.001>",
  "mitre_name": "<nom de la technique>",
  "explanation": "<2-3 phrases en français>",
  "recommended_actions": ["action1", "action2", "action3"],
  "confidence_score": <float 0.0-1.0>,
  "confidence_justification": "<justification>"
}
```

Mécanisme de retry : en cas d'échec du parsing JSON, le système effectue une nouvelle tentative avec un prompt simplifié, puis utilise un JSON de fallback statique (confidence = 0.2, méthode = "fallback").

Formule de confiance pondérée : le score de confiance final est calculé comme une moyenne pondérée de trois sources indépendantes :

Formule de confiance pondérée

$$confidence_{final} = 0.4 \times score_{ML} + 0.3 \times similarity_{RAG} + 0.3 \times score_{LLM}$$

Où $score_{ML} \in [0, 1]$ est le score IsolationForest/LOF, $similarity_{RAG} \in [0, 1]$ est la similarité cosinus de la meilleure correspondance MITRE, et $score_{LLM} \in [0, 1]$ est le score de confiance retourné par le LLM dans son JSON.

4.4 Système de Neutralisation Adaptive

Détecter et expliquer ne suffit pas : la plateforme doit aussi pouvoir agir. Le système de neutralisation adaptive est la composante la plus originale de **CyberSentinel-LLM** sur le plan opérationnel, car il implémente un mécanisme IDS/IPS hybride où toute action bloquante est conditionnée à une décision humaine explicite.

Les quatre modes de défense

Table 4.3: Modes de défense du module de neutralisation

Mode	Comportement	Action produite
`off`	Aucune action	allow — passive uniquement
`monitor`	Surveillance seule	monitor — log uniquement
`assist`	Aide humaine (défaut)	review si score $\geq$ 0.60 ; block_temp si règle connue + high
`auto`	Blocage automatique	block_temp si score $\geq$ 0.85, sans validation humaine

Middleware de blocage (main.py) : un middleware FastAPI intercepte chaque requête entrante et retourne HTTP 403 si l'IP source est dans la liste noire mémoire, avec la commande iptables correspondante :

Réponse HTTP 403 — IP bloquée

```
{
  "error": "IP temporairement bloquée",
  "ip": "203.0.113.5",
  "expires_at": "2025-04-09T14:30:00",
  "reason": "known_attack:ssh_failed_login",
  "iptables_command": "iptables -A INPUT -s 203.0.113.5 -j DROP"
}
```

Flux de neutralisation complet :

Flux : Détection → Notification → Blocage

```
1. Alerte créée (score élevé, sévérité high)
2. decide_defense_action() → block_temp recommandé
3. DefenseAction persistée en SQLite
4. SSE → frontend reçoit l'alerte en temps réel
5. BlockingNotification affiche (cas review ET block_temp pending) :
   - IP source + pays de provenance
   - Type d'attaque + technique MITRE
   - Countdown 30 secondes
   - [APPROUVER] / [REJETER]   (raccourci: Esc pour fermer)
6. Analyste approuve → POST /defense/actions/{id}/approve
7. ip_blacklist.add_ip() → liste noire mémoire O(1)
8. Middleware bloque toutes requêtes de cette IP (HTTP 403)
9. Commande iptables affichée pour blocage réseau optionnel
10. Toast de confirmation : « IP bloquee par <analyste> »
```

Les commandes iptables sont générées automatiquement et affichées à l'analyste pour une application manuelle sur le pare-feu réseau :

Commandes iptables générées

```
# Blocage
iptables -A INPUT -s {ip} -j DROP

# Déblocage
iptables -D INPUT -s {ip} -j DROP
```

4.5 Endpoints principaux de l'API

L'API de **CyberSentinel-LLM** expose ses fonctionnalités à travers un ensemble de routes REST regroupées en six domaines cohérents : ingestion des logs (`/ingest`), gestion des alertes et des incidents (`/alerts`, `/incidents`), interface conversationnelle et génération LLM (`/chat`, `/llm`), neutralisation défensive (`/defense`), calibration ML (`/metrics`), et diffusion d'événements en continu par Server-Sent Events (`/stream`). Le Tableau 4.4 en dresse le catalogue complet.

Table 4.4: Endpoints principaux de l'API CyberSentinel-LLM

Méthode	**Endpoint**	**Rôle**
GET	/health	État API + DB + Ollama + RAG + GeoLite2
POST	/ingest/logs	Ingérer logs, scorer, GeoIP, défense, SSE
GET	/alerts	Lister alertes (filtres status, severity)
POST	/incidents/from-alerts	Créer incident depuis liste d'alertes
POST	/llm/incidents/{id}/explain	Explication LLM + RAG d'un incident
POST	/llm/chat	Chat global streaming
GET	/stream/alerts	Flux SSE temps réel
GET	/map/points	Points GeoIP enrichis
POST	/demo/generate	Génération 3 scénarios SSH simulés
GET	/metrics/model	Métriques ML (F1/précision/rappel/ROC)
POST	/defense/actions/{id}/approve	Approuver blocage → liste noire
POST	/defense/actions/{id}/revoke	Révoquer blocage
GET	/defense/blocked	Liste IPs actuellement bloquées
POST	/defense/block-manual	Blocage manuel d'une IP
POST	/validation/alerts/{id}	Labelliser une alerte

4.6 SSE et communication temps réel

Le flux temps réel repose sur le protocole **Server-Sent Events (SSE)**, qui permet au backend de pousser des événements vers le navigateur sans que le client ait à redemander constamment. Concrètement,

le serveur maintient une collection de files `asyncio.Queue` — une par client actif. Dès qu'une nouvelle alerte est ingérée, `publish_alert_created()` la dépose dans chaque file et le flux SSE la retransmet à tous les abonnés connectés. Un plafond de 256 éléments par file prévient tout engorgement mémoire en cas de rafale d'événements.

4.7 Architecture frontend temps réel

Côté client, toute la logique de flux temps réel et de rafraîchissement périodique est centralisée dans un **contexte React unique** baptisé `RealtimeProvider`. Ce choix évite que chaque page ouvre sa propre connexion SSE ou déclenche ses propres requêtes de polling, ce qui multiplierait inutilement les connexions réseau. Le provider assume quatre responsabilités distinctes :

- ☞ **Une connexion SSE partagée** vers `/stream/alerts`, redistribuée à tous les abonnés via un bus interne (`subscribeToAlerts`). Les pages *Dashboard*, *Alertes*, *Démo* et le composant `AlertPulse` s'y branchent sans jamais instancier leur propre `EventSource` ;
- ☞ **Un polling de défense** vers `/defense/actions` toutes les 5 secondes, qui rafraîchit la liste des actions en attente d'une décision humaine. Cette liste pilote simultanément la `BlockingNotificatio` et un badge pulsant sur l'entrée *Alertes* de la barre de navigation ;
- ☞ **Un polling de santé** vers `/health` toutes les 15 secondes, qui renseigne l'indicateur *Backend connecté / SSE actif* affiché en pied de sidebar ;
- ☞ **Une API de notifications éphémères** (`pushToast`), consommée par l'ensemble des pages pour signaler les confirmations de blocage, les révocations et les erreurs de manière non bloquante.

L'arbre du shell applicatif suit le schéma : `ErrorBoundary > RealtimeProvider > {Sidebar, Pages, BlockingNotification, ToastStack}`.

Ergonomie Human-in-the-Loop. Le bouton *Approuver* est désormais actif aussi bien pour les actions `review` que pour les actions `block_temp` en attente, rendant visible et obligatoire le choix de l'analyste avant toute mise en liste noire. Les opérations irréversibles — réinitialisation de la démo, purge de la base — sont protégées par une boîte de dialogue modale (raccourcis Échap / Entrée). Durant la phase de génération LLM (30 à 90 secondes sur CPU), un panneau d'avancement avec barre de progression informe l'opérateur que le système travaille, évitant toute confusion avec un gel de l'interface.

Résilience. Un `ErrorBoundary` React enveloppe l'intégralité de l'arbre de composants : toute exception non interceptée déclenche un écran de récupération (« Réessayer / Recharger ») plutôt qu'un écran blanc, ce qui est indispensable pour une console SOC devant rester opérationnelle en toutes circonstances.

4.8 Interfaces utilisateur

Les captures d'écran ci-après donnent à voir les principales pages de l'interface SOC de **CyberSentinel-LLM**, dans l'ordre du flux opérationnel typique d'un analyste.

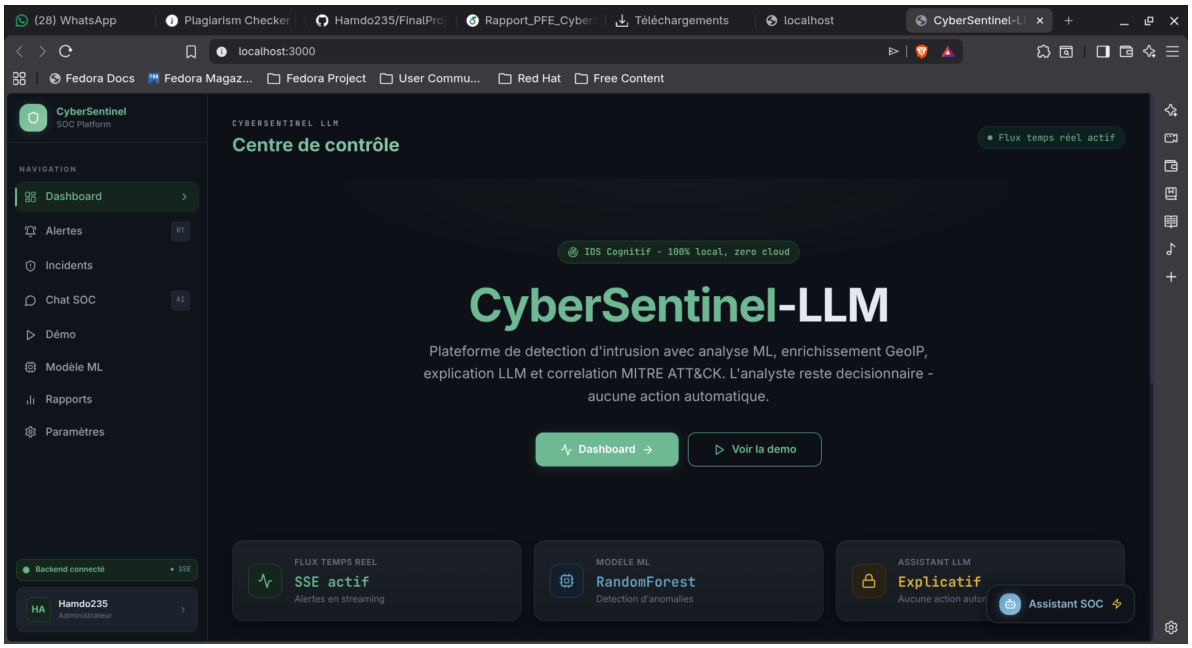


Figure 4.1: Tableau de bord SOC — KPIs temps réel, timeline 24h et carte GeoIP

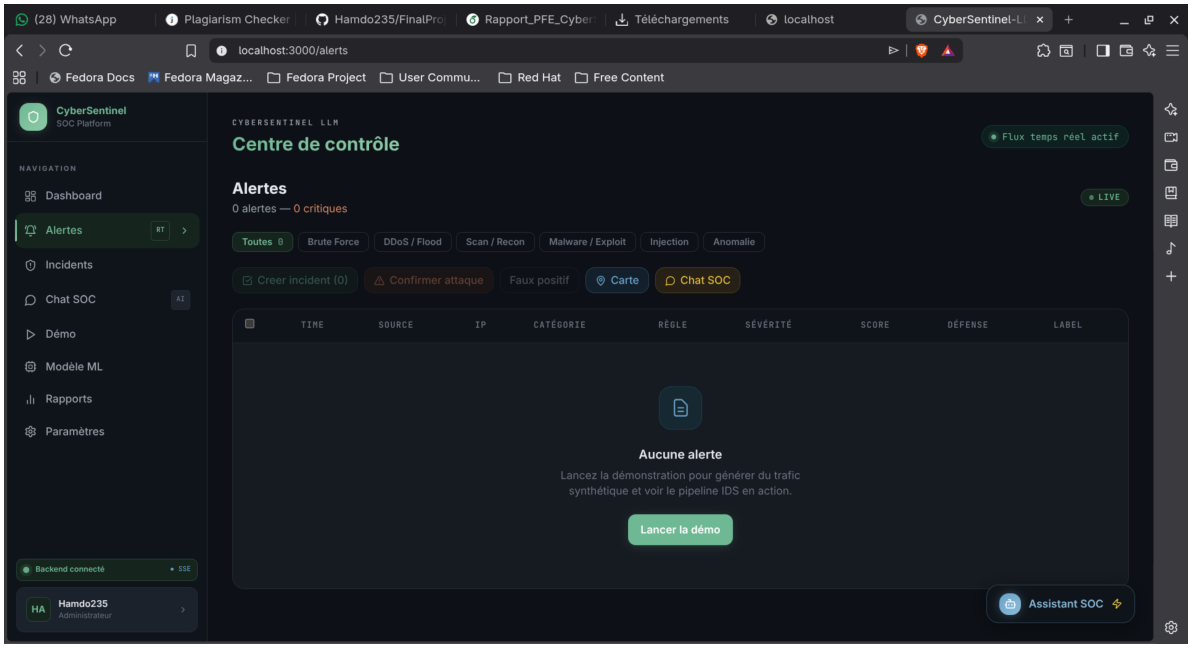


Figure 4.2: Page Alertes — liste complète avec labellisation et gestion des blocages

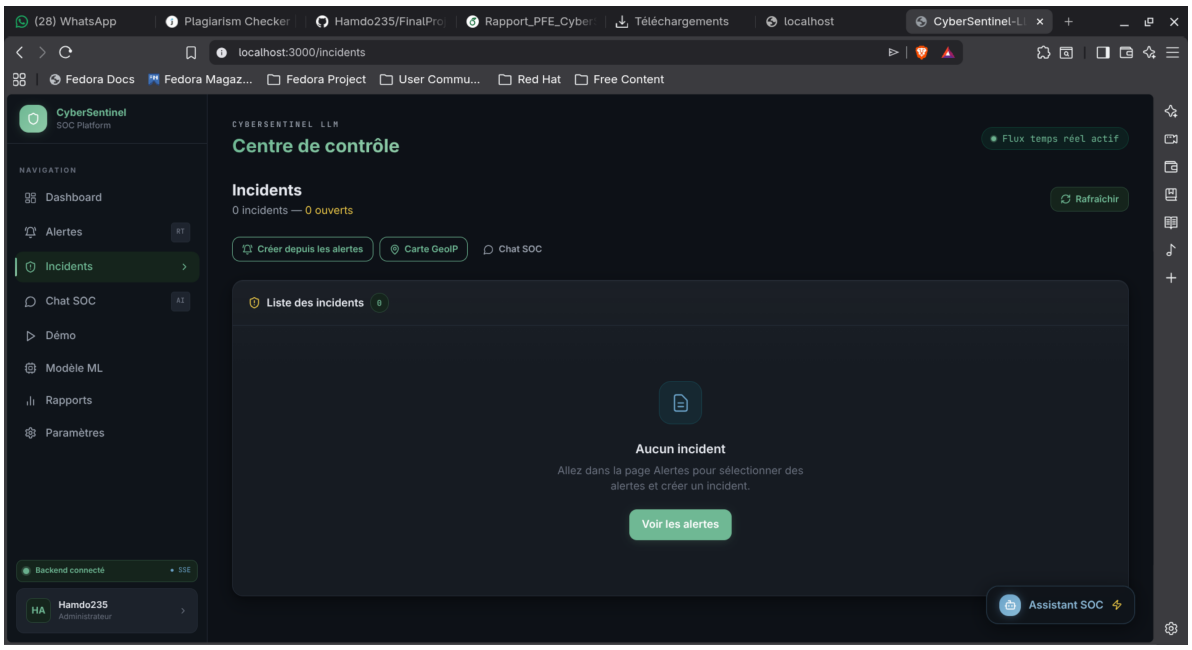


Figure 4.3: Page Incidents — détail d'un incident avec explication LLM structurée

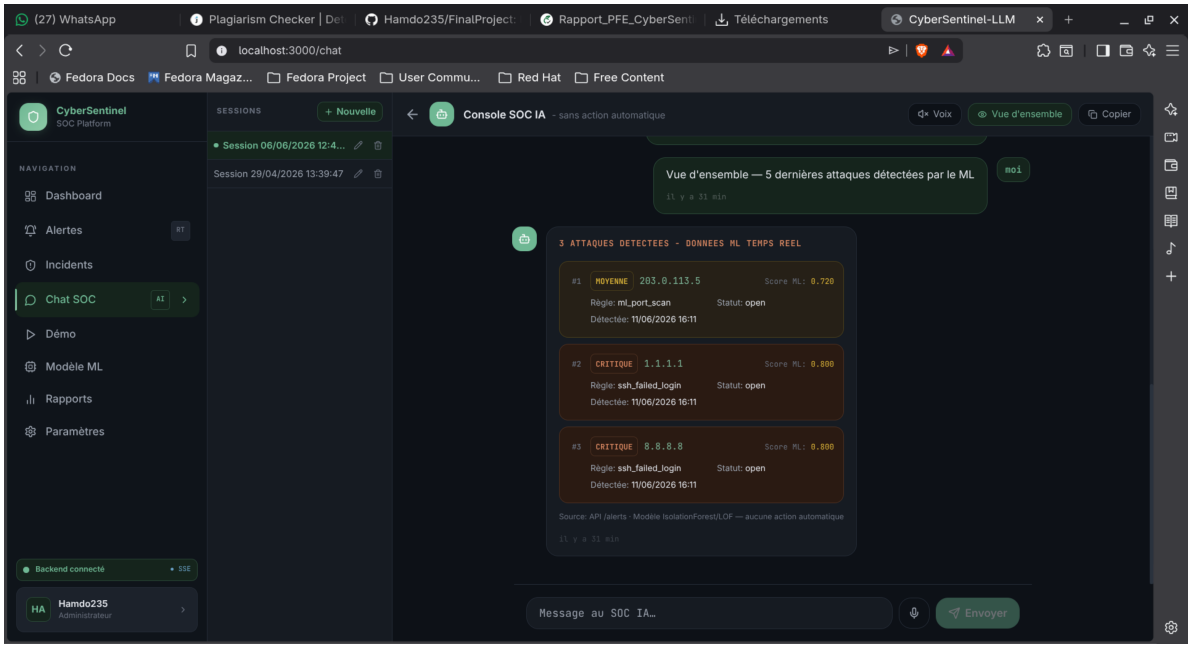


Figure 4.4: Page Chat SOC — console conversationnelle streaming avec historique

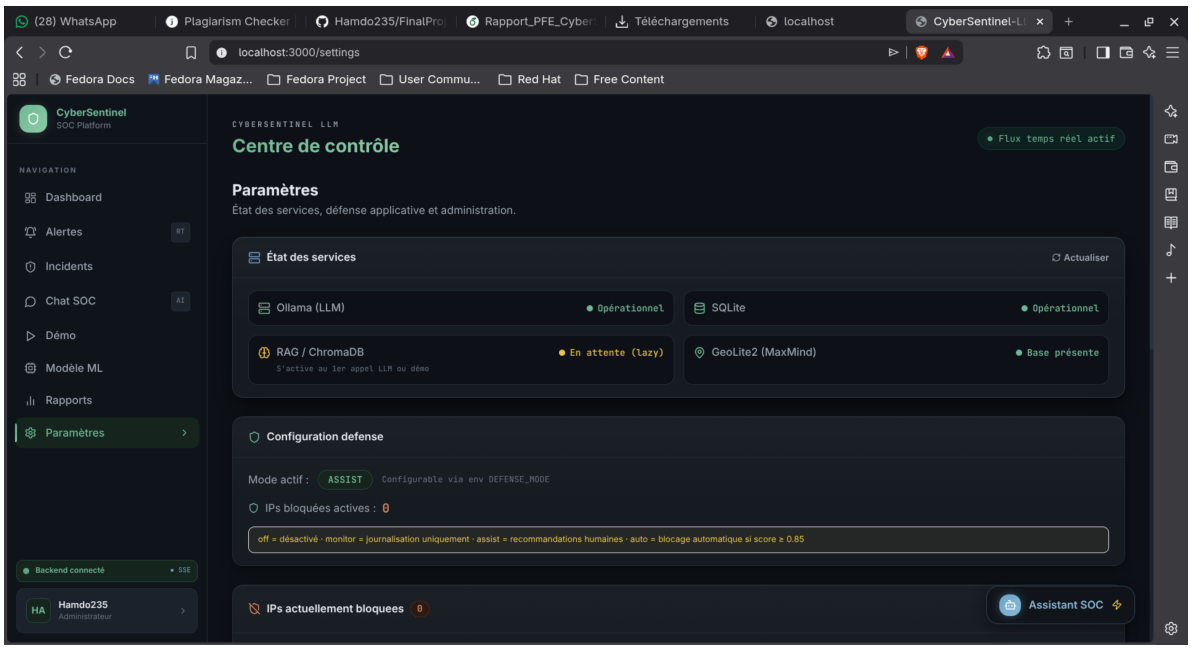


Figure 4.5: Page Paramètres — état des services, 2FA TOTP, gestion des utilisateurs et IPs bloquées

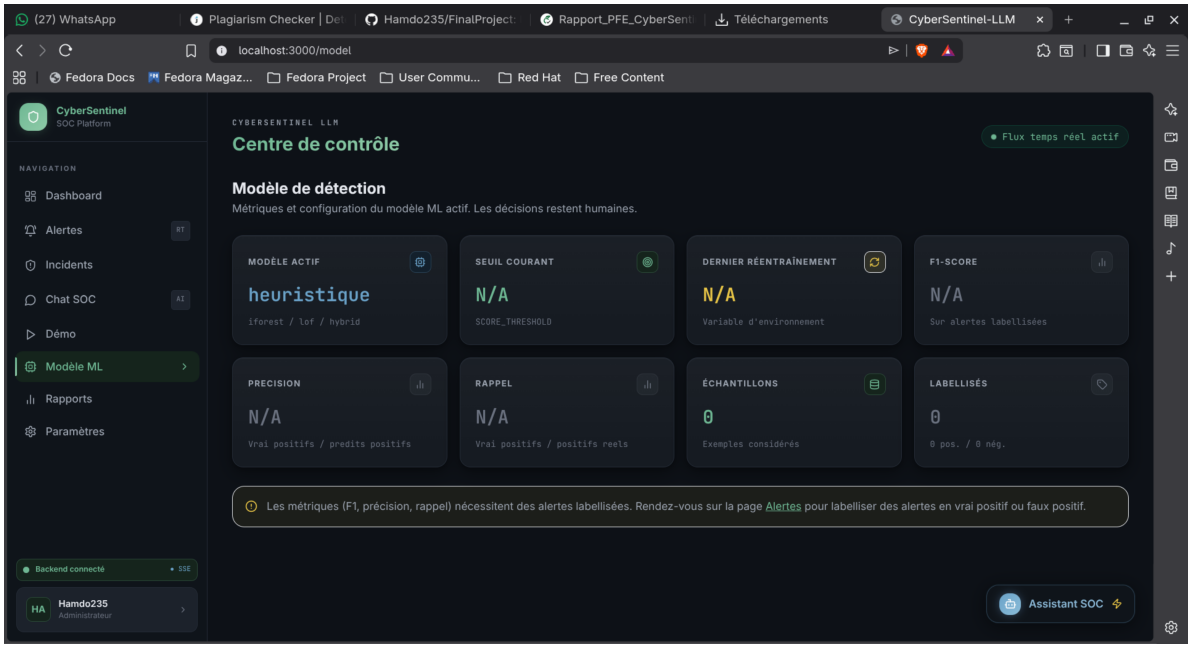


Figure 4.6: Page Modèle ML — métriques F1-score, précision, rappel et ROC-AUC

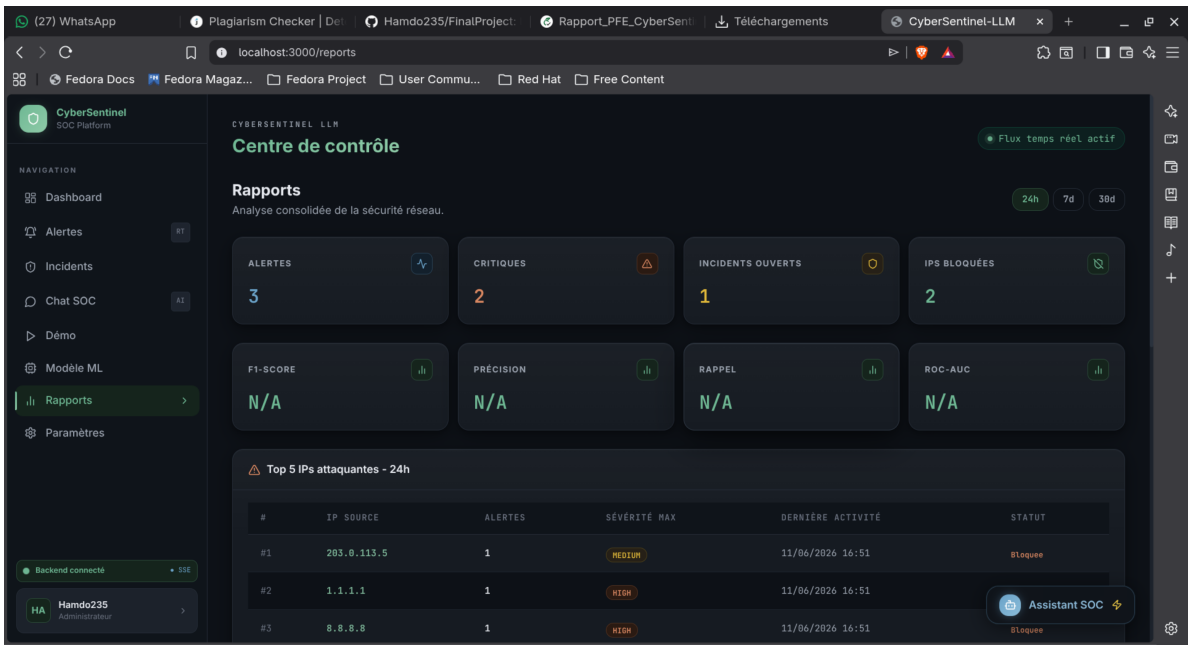


Figure 4.7: Page Rapports — statistiques par période, top attaquants et export CSV

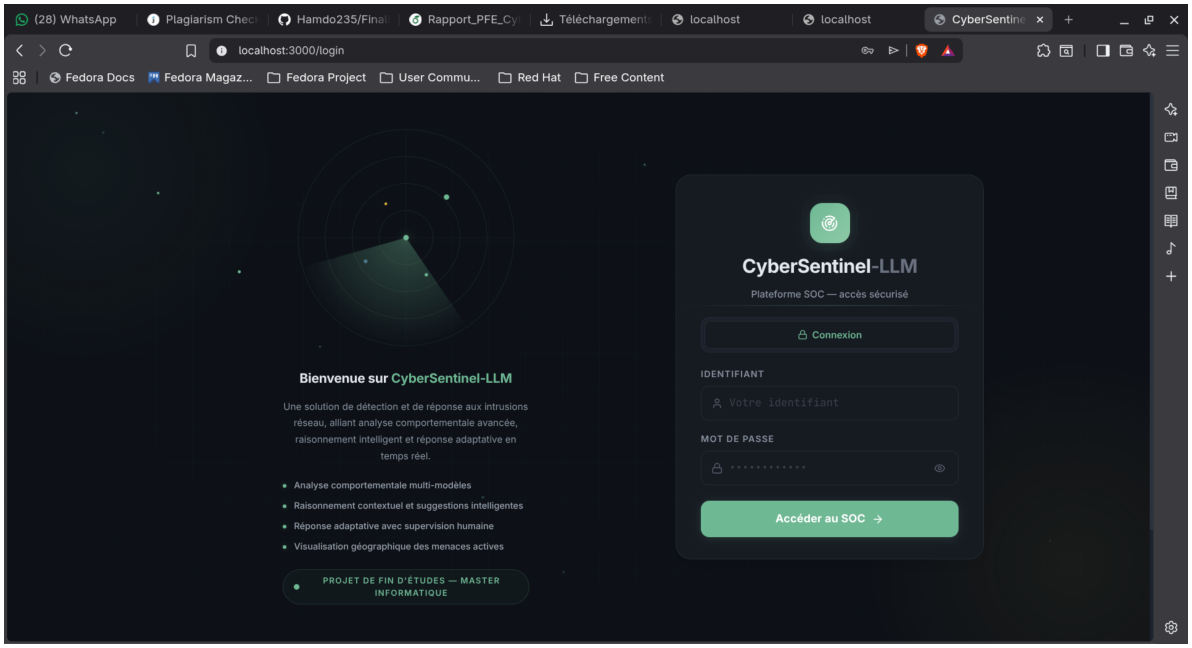


Figure 4.8: Page Connexion — authentification et inscription avec gestion des sessions

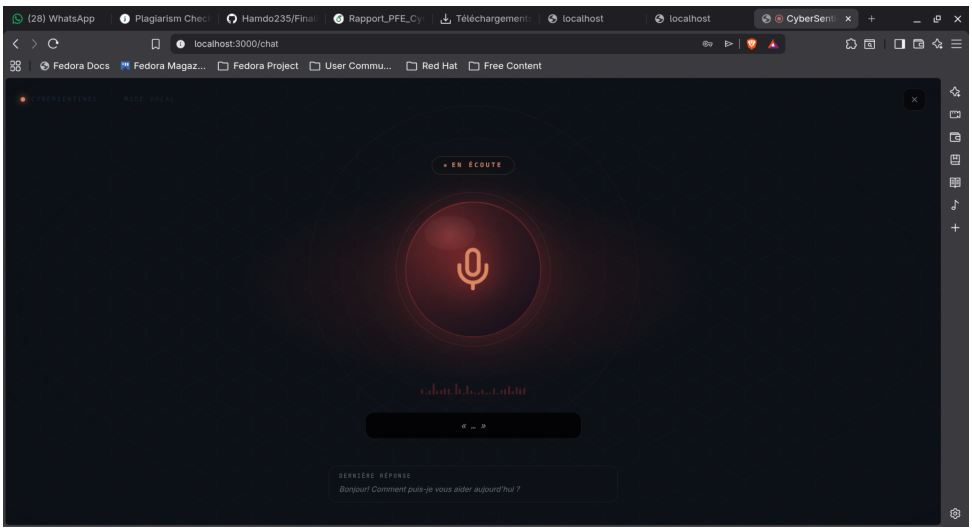


Figure 4.9: Overlay mode vocal — orb animée, équaliseur et transcription en temps réel

Conclusion

Ce chapitre a décortiqué le fonctionnement interne de chaque brique de **CyberSentinel-LLM** : les formules de scoring ML et leurs modes de fallback, la mécanique d'interrogation RAG et d'embedding vectoriel, le template de prompt et le calcul de confiance pondérée, le flux de neutralisation applicative avec notification humaine, et l'organisation des endpoints REST. Le chapitre suivant évalue concrètement les résultats obtenus lors des tests du prototype.

CHAPTER 5

TESTS, VALIDATION ET RÉSULTATS

Introduction

Un prototype ne vaut que par ce qu'il produit lorsqu'on le fait tourner. Ce chapitre rend compte des tests conduits sur **CyberSentinel-LLM** : le scénario de démonstration reproductible conçu pour exercer le pipeline de bout en bout, les métriques retenues pour évaluer le module ML, et les résultats observés — avec leurs forces et leurs limites assumées.

5.1 Scénario de démonstration complet

Pour vérifier la cohérence du système de bout en bout, un scénario de démonstration entièrement reproductible a été développé sur la page `/demo`. Il simule une intrusion réelle en cinq étapes enchaînées :

1. **Génération des logs** — `POST /demo/generate` : 3 événements réseau sont créés : 2 alertes SSH échouées depuis des IPs externes (US, AU) à sévérité HIGH avec recommandation `block_temp`, et 1 requête HTTP suspecte depuis 203.0.113.5 (JP) à sévérité MEDIUM avec action `review` (supervision humaine requise). Les IPs sont automatiquement géolocalisées (US, AU, JP) ;

2. **Détection ML** — affichage du résultat du scoring : "Critique: 2, Faible: 1 — Score max: 0.800" ;

3. **Enrichissement GeoIP** — 3 points apparaissent sur la carte Leaflet (CartoDB dark matter) ;

4. **Analyse LLM + RAG** — incident créé, `retrieve_context()` identifie T1110.001, T1110.003, T1078 comme techniques pertinentes. Ollama génère : *"Brute Force SSH — T1110.001 — confidence: 0.74"* ;

5. **Neutralisation** — approbation automatique des 2 blocages HIGH en démo, commandes iptables affichées, middleware HTTP 403 actif ; la notification de blocage est déclenchée pour les alertes HIGH (`block_temp`) et MEDIUM avec action `review` (supervision humaine requise).

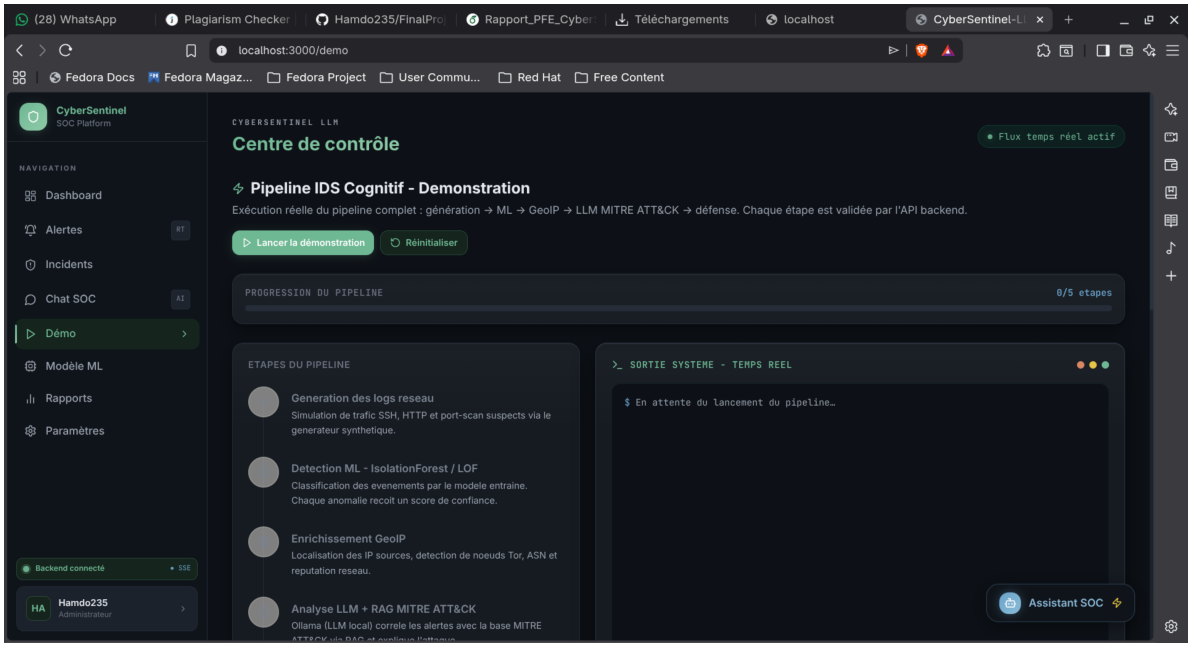


Figure 5.1: Page Démo — pipeline IDS 5 étapes avec terminal live et carte GeoIP

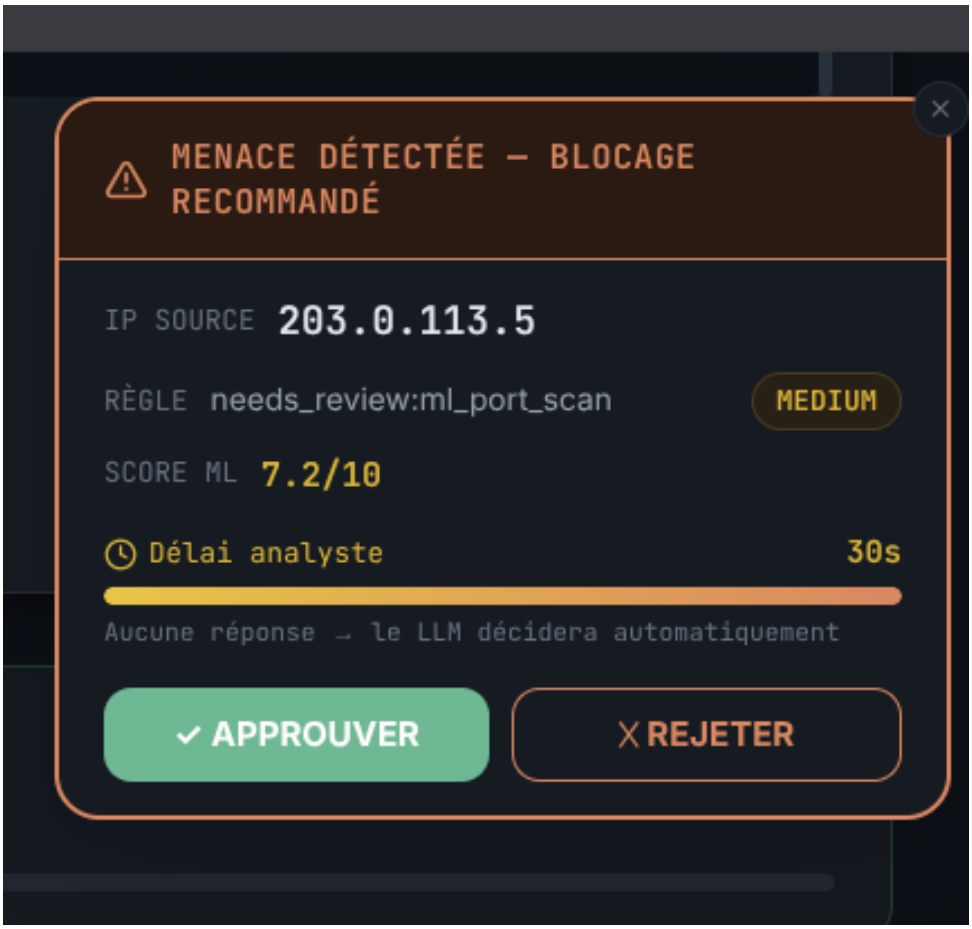


Figure 5.2: Notification de blocage — countdown 30s, détails de l'attaque et validation humaine

5.2 Mécanismes d'évaluation

La qualité du module de détection est mesurée à partir des labels que l'analyste attribue à chaque alerte (`true_attack` ou `false_positive`). Ces annotations alimentent le calcul des quatre métriques standard présentées dans le Tableau 5.1.

Table 5.1: Métriques d'évaluation du module de détection

Métrique	**Formule**	**Interprétation**
Précision	$\frac{VP}{VP+FP}$	Proportion d'alertes pertinentes parmi celles émises
Rappel	$\frac{VP}{VP+FN}$	Proportion d'attaques réelles détectées
F1-Score	$\frac{2\times P\times R}{P+R}$	Moyenne harmonique précision/rappel
ROC-AUC	Aire sous la courbe ROC	Performance globale du classifieur
Confiance pondérée	$0.4\times s_{ML}+0.3\times s_{RAG}+0.3\times s_{LLM}$	Fiabilité globale de l'analyse

5.3 Résultats observés

L'exécution du scénario de démonstration et des tests unitaires a permis de valider les comportements suivants :

- ☞ **Pipeline ML :** scoring d'anomalie fonctionnel avec un temps de traitement inférieur à 50 ms par log en mode hybride (IsolationForest + LOF + heuristique) ;
- ☞ **RAG MITRE ATT&CK :** pertinence élevée des techniques récupérées pour le scénario SSH (T1110.001, T1110.003, T1078 correctement identifiées parmi 697 techniques indexées) ;
- ☞ **Explications LLM :** cohérence et structuration satisfaisantes des rapports générés par Ollama (*qwen2.5:1.5b*), avec un temps de génération compris entre 30 et 90 secondes selon la complexité de l'incident ;
- ☞ **Système de blocage :** bon fonctionnement du mécanisme de neutralisation applicative (middleware HTTP 403 + commandes iptables) avec validation humaine obligatoire avant toute activation ;
- ☞ **Streaming SSE :** stabilité du flux temps réel confirmée avec plusieurs abonnés simultanés sur `/stream/alerts` sans perte d'événements observée ;
- ☞ **Tests automatisés :** ensemble des tests pytest (logique de défense, liste noire IP) et Vitest (authentification, 2FA) passants, sans régression détectée.

Table 5.2: Synthèse des fonctionnalités — état final

Fonctionnalité	**Statut**	**Remarque**
Ingestion + scoring ML	Complet	IF/LOF/hybride fonctionnels
RAG MITRE ATT&CK (ChromaDB)	Complet	697 techniques indexées (STIX Enterprise)
Explication LLM structurée JSON	Complet	Retry + fallback
Confiance pondérée (ML+RAG+LLM)	Complet	Formule transparente
SSE temps réel	Complet	Multi-abonnés, backpressure
Middleware blocage HTTP 403	Complet	Liste noire O(1)
Notification countdown 30s	Complet	Human-in-the-Loop
Approbation `block_temp` pending	Complet	Choix explicite analyste
Commandes iptables générées	Complet	Affichage uniquement
Provider temps réel centralisé	Complet	SSE + polling unifiés
Indicateur santé backend / SSE	Complet	Sidebar dynamique
Badge actions en attente	Complet	Compteur pulsant Alertes
Toasts & confirmations modales	Complet	Reset démo / DB protégés
Error Boundary global	Complet	Récupération sans rechargement
Enrichissement GeoIP	Partiel	Nécessite GeoLite2-City.mmdb
Chat SOC + mode voix	Partiel	STT/TTS si modèles présents
Métriques ML (F1, etc.)	Partiel	Nécessite labels humains
Authentification 2FA TOTP	Complet	QR code + TOTP, côté frontend
Gestion des utilisateurs (CRUD)	Complet	localStorage, rôles, mot de passe
Tests automatisés backend	Complet	pytest : 6 fichiers de tests
Tests automatisés frontend	Complet	Vitest : auth.test.ts
Refactoring modulaire frontend	Complet	Chat : -37%, Settings : modulaire
Authentification RBAC backend	× Absent	JWT côté serveur non implémenté
Blocage iptables automatique	× Absent	Affichage manuel uniquement

5.4 Tests automatisés

Pour sécuriser les composants critiques contre les régressions, une suite de tests automatisés a été mise en place côté backend avec pytest et côté frontend avec Vitest.

Tests backend (pytest)

- ☞ **test_health.py :** tests de santé (`GET /health`) ;
- ☞ **test_ingest_and_map.py :** tests d'ingestion (`POST /ingest/logs`) et de cartographie (`GET /map/points`) ;
- ☞ **test_llm_chat.py :** tests du chat LLM (`POST /llm/chat`) en mode streaming et non-streaming ;
- ☞ **test_security.py :** tests de sécurité (protection admin token, validation TTL, limites batch) ;
- ☞ **test_defense.py :** tests unitaires du service de défense (`services/defense.py`) :
  - `decide_defense_action()` : validation des 4 modes (off, monitor, assist, auto) et des règles de seuil ;
  - `expire_actions()` : désactivation automatique des actions expirées en base SQLite ;
  - Endpoints `POST /defense/actions/{id}/approve` et `POST /defense/auto-decide`
- ☞ **test_ip_blacklist.py :** tests unitaires du service de liste noire IP (`services/ip_blacklist.py`) :
  - `add_ip()` : ajout avec expiration, raison, approbateur ;
  - `is_blocked()` : vérification O(1) avec purge auto des entrées expirées ;
  - `remove_ip()` : retrait thread-safe ;
  - `get_all()` : liste complète avec purge ;
  - `expire_loop` : boucle d'expiration automatique.

Tests frontend (Vitest)

- ☞ **Configuration Vitest :** fichier `vitest.config.ts` avec environnement `happy-dom`, mocks pour `next/navigation` et `next/dynamic` ;
- ☞ **Scripts package.json :** `npm test` (exécution) et `npm run test:ui` (interface visuelle) ;
- ☞ **auth.test.ts :** tests complets d'authentification et 2FA (`components/layout/AuthProvider`) :
  - `addUser()` : création utilisateur avec validation ;
  - `login()` : vérification mot de passe + token JWT simulé ;
  - `logout()` : nettoyage localStorage ;
  - `generateMfaSecret()` : génération secret base32 ;
  - `getMfaQrUri()` : URI otpauth pour QR code ;

- `verifyMfaToken()` : validation code 6 chiffres ;
- `enableMfa()` / `disableMfa()` : activation/désactivation ;
- `getUsername()` / `getUserProfile()` : gestion sessions.

☞ **Build et lint :** build Next.js (`npm run build`) et lint TypeScript (`tsc -noEmit`) sans erreur bloquante, y compris après l'introduction du provider temps réel centralisé, de l'`ErrorBoundary` global et du système de toasts.

Conclusion

Les résultats de cette campagne de tests valident l'hypothèse centrale du projet : il est techniquement réalisable de coupler un pipeline ML non supervisé, un moteur RAG MITRE ATT&CK et un LLM local pour produire un assistant SOC opérationnel, avec des métriques mesurables et un scénario de démonstration reproductible.

La couverture de tests automatisés — pytest sur la logique de défense et la liste noire IP, Vitest sur l'authentification et le 2FA — constitue un filet de sécurité qui protège les fonctionnalités critiques contre les régressions futures. Le refactoring modulaire du frontend contribue quant à lui à rendre le code plus lisible et plus facile à maintenir sur la durée.

Deux lacunes demeurent cependant : l'authentification JWT côté serveur n'a pas été implémentée, et les commandes iptables sont affichées sans être exécutées automatiquement. Ces points constituent les premières pistes de travail pour la suite du projet.

CONCLUSION GÉNÉRALE ET PERSPECTIVES

Ce mémoire a documenté, de la première esquisse de conception jusqu'aux résultats de tests, le développement de **CyberSentinel-LLM** : une plateforme cognitive de cybersécurité qui articule détection d'anomalies par Machine Learning, enrichissement contextuel par RAG sur MITRE ATT&CK, explication en langage naturel via un LLM local, et neutralisation soumise à validation humaine.

Le prototype réalisé démontre concrètement qu'il est possible, sur du matériel standard et sans cloud, de construire une chaîne de traitement complète en cinq couches : un pipeline ML hybride (IsolationForest + LOF + heuristique), une base vectorielle ChromaDB indexant 697 techniques MITRE, un modèle Qwen2.5 déployé via Ollama qui génère des analyses JSON avec un score de confiance pondéré ($0.4 \times ML + 0.3 \times RAG + 0.3 \times LLM$), un mécanisme de blocage applicatif middleware avec countdown et approbation humaine obligatoire, et une interface Next.js 9 pages en mode sombre.

Le fil conducteur **Human-in-the-Loop** n'a jamais été sacrifié : le LLM n'exécute aucune action autonome sur le système, et chaque mesure défensive ne s'active qu'après un choix explicite de l'opérateur.

Dans un Tchad où les équipes SOC sont en construction et les ressources limitées, **CyberSentinel-LLM** apporte une réponse pragmatique : déployable sur une machine ordinaire, indépendant de toute infrastructure externe, et conçu pour assister des analystes même peu expérimentés.

Sur le plan de la robustesse, le projet embarque une authentification 2FA TOTP compatible avec les applications standard, ainsi qu'une suite de tests automatisés pytest et Vitest couvrant la défense, la liste noire IP et l'authentification frontend.

Limites identifiées : l'architecture monolithique limite la montée en charge, l'authentification JWT côté serveur est absente (les sessions sont gérées en localStorage), les commandes iptables ne sont pas exécutées automatiquement, et le périmètre ML se cantonne actuellement aux logs SSH.

Pistes d'évolution :

- ☞ Passage à un entraînement supervisé multi-classes pour couvrir un spectre plus large de familles d'attaques : DDoS, injection SQL, ransomware et mouvements latéraux ;
- ☞ Élargissement de la base RAG aux CVE récentes et aux bulletins de l'ANSSI, afin de mieux adresser les menaces qui ciblent spécifiquement la région Afrique centrale (la base MITRE ATT&CK Enterprise à 697 techniques est déjà intégralement indexée dans la version actuelle) ;
- ☞ Exécution automatique des règles iptables après validation humaine, via des scripts système isolés et journalisés, pour franchir le dernier kilomètre entre l'affichage et l'action réseau réelle ;
- ☞ Découpage en microservices indépendants et introduction d'un broker d'événements (Redis ou Kafka) pour permettre la montée en charge horizontale du pipeline de détection ;
- ☞ Implémentation d'une authentification JWT côté serveur et d'un contrôle d'accès basé sur les rôles (RBAC) pour sécuriser les opérations critiques au niveau backend ;
- ☞ Ancrage dans le terrain tchadien : déploiement pilote dans un environnement opérationnel réel, formation des analystes locaux, et collaboration avec les programmes de montée en compétences de l'ANSICE.

BIBLIOGRAPHY

[1] **Liu, F. T., Ting, K. M., Zhou, Z.-H.** (2008). Isolation Forest. *IEEE International Conference on Data Mining (ICDM)*, pp. 413–422.

[2] **Breunig, M. M., Kriegel, H.-P., Ng, R. T., Sander, J.** (2000). LOF: Identifying Density-Based Local Outliers. *ACM SIGMOD Record*, 29(2), pp. 93–104.

[3] **Lewis, P., Perez, E., Piktus, A., et al.** (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33.

[4] **Motlagh, F. N., Hajizadeh, M., Majd, M., et al.** (2024). Large Language Models in Cybersecurity: State-of-the-Art. *arXiv preprint arXiv:2402.00891*.

[5] **Houssel, P. R. B., Singh, P., Layeghy, S., & Portmann, M.** (2024). Towards Explainable Network Intrusion Detection using Large Language Models. *arXiv preprint arXiv:2408.04342*.

[6] **Ferrag, M. A., et al.** (2023). SecurityBERT: A Privacy-Preserving BERT-based Lightweight Model for IoT Cybersecurity. *arXiv preprint arXiv:2306.14263*.

[7] **MITRE Corporation** (2024). ATT&CK — Adversarial Tactics, Techniques and Common Knowledge. *https://attack.mitre.org/*

[8] **FastAPI** — Documentation officielle. *https://fastapi.tiangolo.com/*

[9] **Ollama** — Documentation officielle. *https://ollama.com/*

[10] **ChromaDB** — The AI-native open-source embedding database. *https://docs.trychroma.com/*

[11] **Reimers, N., Gurevych, I.** (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*. *https://sbert.net/*

[12] **Next.js 16** — Documentation officielle. *https://nextjs.org/docs*

[13] **Scikit-learn** — Documentation officielle. *https://scikit-learn.org/stable/*

[14] **DataReportal** (2025). Digital 2025: Chad. *https://datareportal.com/reports/digital-2025-chad*

[15] **ANSICE** — Agence Nationale de Sécurité Informatique et de Certification Électronique du Tchad. *https://www.ansice.td/*

ANNEXES

Annexe A : Architecture globale du système

Cette annexe reproduit, pour consultation rapide, le schéma d'architecture globale présenté et commenté à la section 3.1. Il synthétise en un coup d'œil les cinq couches du système et les flux qui les relient.

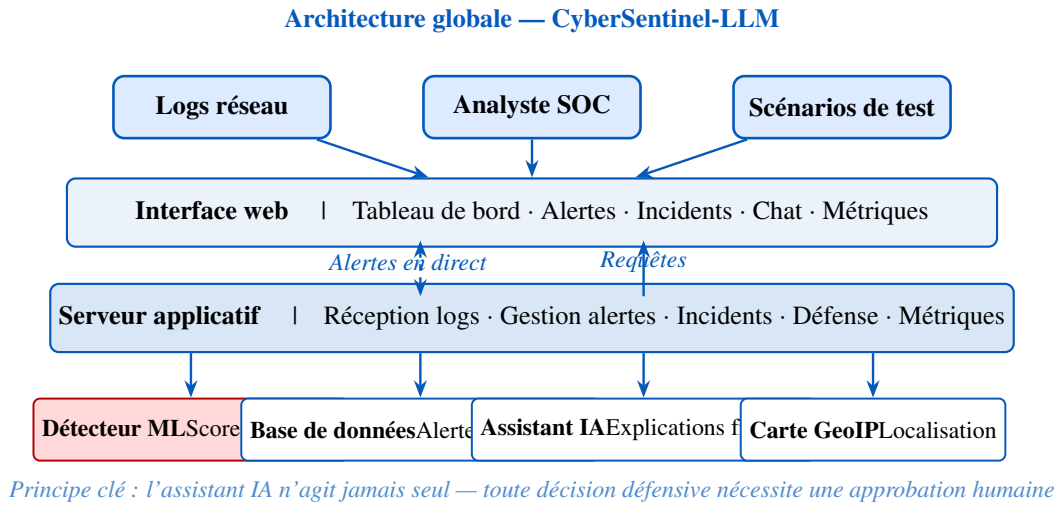


Figure 5.3: Architecture globale de CyberSentinel-LLM

Annexe B : Exemples de payloads API

Les deux échanges HTTP ci-dessous illustrent le format exact des données manipulées par les endpoints les plus critiques : l'ingestion d'un log réseau et l'approbation d'un blocage défensif.

POST /ingest/logs — Corps de requête et réponse générée :

```
// Requête
{
  "ts": "2025-04-09T14:32:00Z",
  "source": "ssh",
  "ip": "185.220.101.45",
  "user": "root",
  "status": "fail"
}

// Réponse -- Alerte créée
{
  "id": 42,
  "ts": "2025-04-09T14:32:00Z",
  "source": "ssh",
  "ip": "185.220.101.45",
  "rule": "ml_iforest_anomaly",
  "severity": "high",
  "score": 0.87,
  "status": "open"
}
```

POST /defense/actions/{id}/approve — Réponse incluant les commandes iptables :

```
{
  "ok": true,
  "message": "IP 185.220.101.45 ajoutée à la liste noire applicative",
  "expires_at": "2025-04-09T15:02:00",
  "iptables_command": "iptables -A INPUT -s 185.220.101.45 -j DROP",
  "iptables_unblock": "iptables -D INPUT -s 185.220.101.45 -j DROP"
}
```

Annexe C : Guide d'installation et d'exécution

Ce guide décrit la procédure minimale pour déployer et démarrer **CyberSentinel-LLM** sur une machine locale. L'ensemble du système fonctionne sans connexion cloud une fois les dépendances installées.

Prérequis système :

- ☞ Python 3.10 ou supérieur avec pip
- ☞ Node.js 18 ou supérieur avec npm
- ☞ Ollama installé avec le modèle qwen2.5:1.5b récupéré via la commande `ollama pull qwen2.5:1.5b`
- ☞ GeoLite2-City.mmdb (facultatif — requis uniquement pour la géolocalisation des IPs)

Démarrage du backend (API FastAPI) :

```
cd backend
pip install -r requirements.txt
bash start.sh
# API disponible sur http://localhost:8000
# Surveille uniquement app/ (pas chroma_db/ ni data/)
# Documentation Swagger : http://localhost:8000/docs
```

Démarrage du frontend (interface Next.js) :

```
cd frontend
npm install
npm run dev
# Interface disponible sur http://localhost:3000
```

Table 5.3: Variables d'environnement principales

Variable	Défaut	Description
OLLAMA_BASE_URL	localhost:11434	URL du serveur Ollama
OLLAMA_MODEL	qwen2.5:1.5b	Modèle LLM utilisé
SCORE_THRESHOLD	0.6	Seuil de détection ML
DETECTOR	iforest	IF / LOF / hybrid
DEFENSE_MODE	assist	off/monitor/assist/auto
DEFENSE_BLOCK_TTL_MINUTES	30	Durée blocage temporaire
DEFENSE_ENFORCE_API	false	Active middleware HTTP 403
GEOLITE2_CITY_DB	(optionnel)	Chemin base GeoIP
APP_ENV	dev	dev ou production